NGUYEN DEVELOPMENT · PHPDBADMIN

Foundation — phpdbadmin

Gestaltung, Wireframes, Solution-Architektur, Datenmodell, PRL, Timebox-Plan

Phase	03 Foundation
Träger	Nguyen Development (Einzelfirma), Hoang Nguyen · Lizenz MIT
Vorgehen	DSDM/AgilePM — Dokumentation in dokumentationen/, ein PDF je Phase
Stand	29.10.2026, 00:47 · baut auf Commit 1c8f969 + Änderungen dieses Standes auf
Quelle	dokumentationen/ im Repo — dieses PDF wird bei jeder Änderung neu gebaut

Dieses Dossier enthält

1. 03 — Foundation
2. Anhang A — Gestaltung — phpdbadmin
3. Anhang B — Wireframes — phpdbadmin
4. Anhang C — Solution-Architektur — phpdbadmin
5. Anhang D — Datenmodell — App-Datenbank v0.1
6. Anhang E — PRL — Prioritised Requirements List (phpdbadmin)
7. Anhang F — Timebox-Plan — phpdbadmin

Inhalt

03 — Foundation

Anhang A — Gestaltung — phpdbadmin

1. Haltung
2. Farben
3. Umgebungen und Rasterzustände
4. Typografie
5. Raster, Abstände, Formen
6. Bausteine
7. Text und Ton
8. Schmale Fenster
9. Bewegung
10. Entscheide (Annahme bis zur Sichtprüfung mit TB 00)

Anhang B — Wireframes — phpdbadmin

Zweck
Aufruf
Bereiche
Bildstrecke (Desktop)
Entscheide
Fragen an Hoang — entschieden 28.10.2026
Grenzen der Wireframes

Anhang C — Solution-Architektur — phpdbadmin

1. Überblick
2. Was die Plattform vorgibt
3. Schichten und Verzeichnisse
4. Anfrageverlauf
5. Zielverbindungen
6. Schreibschutz
7. Katalog
8. Daten lesen und bearbeiten
9. Struktur ändern
10. SQL-Editor
11. Export und Import
12. Anmeldung und Konten
13. Protokoll und Maskierung
14. Qualitätssicherung
15. Entscheide
16. Offene Punkte

Anhang D — Datenmodell — App-Datenbank v0.1

1. Grundsätze
2. Tabellen
3. Was bewusst fehlt
4. Löschen und Aufräumen
5. Offene Punkte

Anhang E — PRL — Prioritised Requirements List (phpdbadmin)

1. Kennzahlen je Stufe
2. Anforderungen je Timebox
3. Regeln

Anhang F — Timebox-Plan — phpdbadmin

1. Die dreizehn Timeboxes
2. TB 00 in Scheiben
3. Meilensteine und Abnahme
4. Reihenfolge und Abhängigkeiten
5. Was aus der Feasibility übernommen wird
6. Protokoll je Timebox (Vorlage)

03 — Foundation

PDF der Phase: foundation.pdf — bei jeder Änderung neu gebaut (`python3 dokumentationen/werkzeuge/pdf-bauen.py --nur 03`).

Status: ✅ **Foundation v1.0 abgeschlossen (28.10.2026)** — begonnen nach der Verabschiedung der Terms of Reference v1.0 (Hoang: «Ja, Foundation»). Review am selben Tag interaktiv entschieden: Schreibschutz auf Zeit als **V-11**, Raster **endlos scrollen**, **PHP 8.5** (ADR-011), Wireframe-Fragen F-2 bis F-6 wie empfohlen (Rückfragen § 5). Die Gestaltung ist mit TB 00 im Browser umgesetzt und gegen die Wireframes 1–4 und 13 bei 1440 und 375 px, hell und dunkel, durchgegangen (Protokoll TB 00); offen bleibt Hoangs eigene Sichtprüfung (G-01 bis G-08).

Dokument	Inhalt
`gestaltung.md`	**v0.1:** Haltung, Farben hell/dunkel mit gerechneten Kontrasten (aus `werkzeuge/kontraste.py`, alle Paare bestehen), Umgebungen und Rasterzustände, Typografie (Inter, JetBrains Mono), Raster und Formen, 14 Bausteine, Text und Ton, schmale Fenster, Entscheide G-01 bis G-08
`mockups/`	**Wireframes v0.1:** 14 Bereiche je Desktop und Mobil aus einem Generator (`mockups/mockup-bauen.py`) — Anmeldung, Code, Verbindungsübersicht, Verbindung anlegen, Arbeitsbildschirm, SQL-Vorschau, Detail-Schublade, Struktur, Tabelle ändern, SQL-Editor, Export/Import, Befehlspalette, Konto, Verwaltung
`solution-architecture.md`	**v1.0:** C4 L2, Plattform (PHP 8.5), Schichten und Verzeichnisse, Anfrageverlauf, Zielverbindungen, Schreibschutz, Katalog, Daten, Struktur, SQL-Editor (Zerleger, Zeilengrenze je Treiber, Abbruch über Merker), Export/Import, Anmeldung, Protokoll und Maskierung, Qualität, Entscheide A-01 bis A-14
`datenmodell.md`	**v0.1:** App-Datenbank — Grundsätze (kein Passwort, verschlüsselt, Prüfsummen, `dateTime`, `longText`, Fristen), 13 Tabellen, was bewusst fehlt, Löschen und Aufräumen
`prl.md`	**v1.0:** 81 Anforderungen mit Timebox, Grösse und Priorität im Plan — erzeugt aus `werkzeuge/prl-bauen.py`; Must-Anteil Stufe 1 57 %, Stufe 2 44 %, bis v1.0 58 %; Stufe 0 bewusst 90 % (Fundament ohne Rückfallweg)
`timebox-plan.md`	**v1.0:** dreizehn Timeboxes TB 00–12 mit Anforderungen, Wireframe-Bereichen, Personentagen (≈ 53 PT); TB 00 in fünf Scheiben; Meilensteine Alpha, v1.0, v1.1, v1.2; Übernahme aus der Feasibility; Protokoll-Vorlage

Architekturentscheide: ADR-001 bis ADR-011 — ausformuliert und angenommen 28.10.2026 (ADR-011 PHP 8.5 von Hoang entschieden).

Diagramme (`diagrams/`, aus `werkzeuge/diagramme.py`): C4 L2 `architektur-l2` · Ändern mit Vorschau `anfrageverlauf` · ER `er-app-datenbank` · Rollen `use-cases` · Export `export-im-hintergrund` · Timebox-Plan `timebox-plan`; C4 L1 und Schichten in CR-001, Anmeldung im Pre-Project, Anwenden je Treiber und Schreibschutz in der Feasibility.

Anhang A — Gestaltung — phpdbadmin

Stand: 28.10.2026 · v0.1 — Vorschlag der Foundation; die Entscheide G-01 bis G-08 gelten als Annahme bis zur Sichtprüfung am echten Bildschirm (die Fragen aus den Wireframes hat Hoang am 28.10.2026 wie empfohlen entschieden). **Umgesetzt mit TB 00:** Palette hell und dunkel im Stylesheet, Inter und JetBrains Mono mit ausgeliefert, Wortmarke, Umgebungszeichen; durchgegangen gegen die Wireframes 1–4 und 13 (Protokoll). Noch nicht umgesetzt: Dichte (G-04, mit dem Raster in TB 01). Hoangs eigene Sichtprüfung steht aus. Farben und Kontraste stehen nur in `werkzeuge/kontraste.py`; die Tabelle in § 2 wird daraus eingesetzt (`python3 dokumentationen/werkzeuge/kontraste.py --schreiben`).

1. Haltung

phpdbadmin ist ein Arbeitswerkzeug für Stunden am Stück. Die Oberfläche tritt zurück, die Daten stehen vorn. Daraus folgen vier Regeln:

1. **Ruhig:** Grund und Flächen in Schiefergrau, ein einziger Akzent (Petrol) für Handlungen, Fokus und Auswahl. Farbe trägt Bedeutung, nie Schmuck.
2. **Ehrlich:** Gefahr hat eine eigene Farbe (Rot) und wird nie für etwas anderes verwendet. Die Umgebung einer Verbindung ist immer sichtbar.
3. **Dicht, aber lesbar:** Daten in tabellarischen Ziffern, Zeilen 32 px, Abstände im 4-px-Raster; nichts wird kleiner als 12 px.
4. **Gleich für alle vier:** Ein Bildschirm sieht für MySQL, MariaDB, PostgreSQL und SQLite gleich aus; der Treiber erscheint als kleines Schild, nicht als eigene Gestaltung.

Abgrenzung: Basalto (Basalt und Lava, Sechseck, IBM Plex) und Fristo haben eigene Identitäten; phpdbadmin übernimmt von dort nur Regeln (Kontraste rechnen, keine Scrollbalken, Sie-Form), keine Formen und Farben.

2. Farben

Die Namen werden als CSS-Variablen auf die Variablen des Starter-Kits gelegt (`--background`, `--foreground`, `--primary`, `--destructive` …) und um eigene ergänzt (`--env-local`, `--env-test`, `--env-production`, `--cell-changed`, `--cell-new`, `--cell-deleted`).

Name	hell	dunkel
`grund`	`#ffffff`	`#0b1120`
`flaeche`	`#f6f7f9`	`#111a2c`
`flaeche-stark`	`#eceff3`	`#1a2438`
`rand`	`#dde2e8`	`#2a3550`
`text`	`#1c2534`	`#e5e9f0`
`text-zweit`	`#4a5568`	`#a9b4c6`
`text-leise`	`#5f6b7d`	`#8c98ad`
`akzent`	`#0b6e82`	`#4fc3d9`
`akzent-text`	`#ffffff`	`#062a33`
`akzent-hell`	`#e3f2f5`	`#0f3540`

Name	hell	dunkel
`erfolg`	`#1d7a46`	`#5bd08f`
`warnung`	`#9a5b00`	`#f2b84b`
`gefahr`	`#b3261e`	`#ff8a80`
`gefahr-text`	`#ffffff`	`#3b0a07`
`gefahr-hell`	`#fbeaea`	`#3a1416`
`umgebung-lokal`	`#4a5568`	`#a9b4c6`
`umgebung-test`	`#9a5b00`	`#f2b84b`
`umgebung-produktion`	`#b3261e`	`#ff8a80`
`zelle-geaendert`	`#e3f2f5`	`#0f3540`
`zelle-neu`	`#e6f4ea`	`#103522`
`zelle-geloescht`	`#fbeaea`	`#3a1416`

Vordergrund auf Hintergrund	Verwendung	Grenze	hell	dunkel
`text` auf `grund`	Fliesstext, Zellwerte	7:1	15,40 ✅	15,46 ✅
`text` auf `flaeche`	Text im Navigator und in Leisten	7:1	14,37 ✅	14,28 ✅
`text-zweit` auf `grund`	Beschriftungen, Spaltentypen	4,5:1	7,53 ✅	8,99 ✅
`text-leise` auf `grund`	NULL, Hilfetexte	4,5:1	5,40 ✅	6,46 ✅
`text-leise` auf `flaeche`	Hilfetexte auf Flächen	4,5:1	5,04 ✅	5,97 ✅
`akzent` auf `grund`	Links, Fokus, aktive Reiter	4,5:1	5,89 ✅	9,09 ✅
`akzent-text` auf `akzent`	Text auf Hauptknopf	4,5:1	5,89 ✅	7,31 ✅
`text` auf `akzent-hell`	Text in geänderter Zelle	7:1	13,41 ✅	10,75 ✅
`erfolg` auf `grund`	Erfolgsmeldung	4,5:1	5,35 ✅	9,74 ✅
`warnung` auf `grund`	Warnung, Umgebung Test	4,5:1	5,43 ✅	10,52 ✅
`gefahr` auf `grund`	Fehler, Umgebung Produktion	4,5:1	6,54 ✅	8,25 ✅
`gefahr-text` auf `gefahr`	Text auf Gefahr-Knopf	4,5:1	6,54 ✅	7,46 ✅
`gefahr` auf `gefahr-hell`	Produktions-Chip, Fehlerkarte	4,5:1	5,62 ✅	7,12 ✅
`text` auf `zelle-neu`	Text in neuer Zeile	7:1	13,56 ✅	11,09 ✅
`text` auf `zelle-geloescht`	Text in gelöschter Zeile	7:1	13,24 ✅	13,35 ✅
`rand` auf `grund`	Rasterlinien (Schmuck, keine Information)	1,3:1	1,30 ✅	1,55 ✅
`akzent` auf `flaeche`	Fokusrahmen (Nicht-Text, WCAG 1.4.11)	3:1	5,49 ✅	8,39 ✅

Zuordnung zu den Variablen des Starter-Kits: `grund` → `--background`, `--card`, `--popover` · `text` → `--foreground` · `akzent` → `--primary`, `--ring` · `akzent-text` → `--primary-foreground` · `flaeche` → `--muted`, `--secondary`, `--sidebar-background` · `text-leise` → `--muted-foreground` · `rand` → `--border`, `--input` · `gefahr` → `--destructive` · `gefahr-text` → `--destructive-foreground`.

Regeln. Akzent nur für eine Handlung je Bereich (der Hauptknopf), für Fokus und für die aktuelle Auswahl. Gefahr nur für zerstörende Handlungen, Fehler und die Umgebung «Produktion». Farbe nie allein: Jede farbige Aussage hat auch ein Wort oder ein Symbol (WCAG 1.4.1).

3. Umgebungen und Rasterzustände

Zustand	Darstellung	Wo
Umgebung **lokal**	Schiefer, Punkt ○	Verbindungs-Chip, Reiter, Seitentitel-Präfix fehlt
Umgebung **Test**	Bernstein, Punkt ◐, Präfix «TEST ·» im Seitentitel	dito
Umgebung **Produktion**	Rot, Punkt ●, Präfix «PROD ·», rotes Favicon, Schloss-Symbol wenn schreibgeschützt	dito; zusätzlich roter 2-px-Streifen am oberen Rand
Zelle **geändert**	Fläche `zelle-geaendert`, Wert fett, kleiner Punkt in der Ecke	Datenraster, bis «Anwenden» oder «Verwerfen»
Zeile **neu**	Fläche `zelle-neu`, Zeilennummer «+»	dito
Zeile **zum Löschen**	Fläche `zelle-geloescht`, Werte durchgestrichen, Zeilennummer «–»	dito
Wert **NULL**	`NULL` kursiv in `text-leise` — nie leer, damit NULL und leerer Text unterscheidbar bleiben	überall
Fremdschlüssel	Pfeil → vor dem Wert, Klick folgt dem Verweis	Datenraster

4. Typografie

Rolle	Schrift	Grösse / Zeile	Hinweis
Oberfläche	**Inter** (OFL 1.1)	14 px / 20 px, Überschriften 16–20 px	`font-variant-numeric: tabular-nums` in Rastern und Zahlen
Daten im Raster	Inter, tabellarische Ziffern	13 px / 32 px Zeile (kompakt 28 px)	lange Werte abgeschnitten mit «…», ganz in der Detail-Schublade
SQL, DDL, Bezeichner	**JetBrains Mono** (OFL 1.1)	13 px / 20 px	Editor, Vorschau, Typnamen
Kleinste Schrift	Inter	12 px	nur Schilder und Zähler

Beide Schriften werden wie heute Instrument Sans über `laravel-vite-plugin/fonts` im Build mitgeliefert — keine Abfrage an einen Schriftendienst zur Laufzeit. Die OFL-Lizenz der Schriften verträgt sich mit der MIT-Lizenz des Codes (die Schriftdateien behalten ihre Lizenz).

5. Raster, Abstände, Formen

- **4-px-Raster**; Innenabstände 8/12/16 px, Abstand zwischen Bereichen 24 px.
- **Radien:** 6 px für Felder und Knöpfe, 10 px für Karten und Schubladen, 999 px für Chips.
- **Linien:** 1 px `rand`; der Fokusrahmen ist 2 px `akzent` mit 2 px Abstand und immer sichtbar bei Tastaturbedienung.
- **Schatten:** nur für schwebende Ebenen (Befehlspalette, Menüs, Schubladen), sonst flach.
- **Symbole:** Lucide (im Starter-Kit installiert), 16 px in Leisten, 20 px in Leerzuständen; Symbole stehen nie allein ohne Beschriftung oder Tooltip.

6. Bausteine

Baustein	Verhalten
Kopfleiste	Verbindungs-Chip (Umgebung, Name, Treiber-Schild, Schloss bei Schreibschutz) · Befehlspalette-Feld mit ⌘K · Kontomenü
Navigator	Baum Verbindung → Datenbank → Schema → Gruppe → Objekt; Suche oben; Gespeichertes und Verlauf unten; auf schmalen Fenstern als Schublade
Reiter	Daten · Struktur · Beziehungen · SQL je Objekt; offene Objekte als Reiterleiste, die umbricht statt zu scrollen
Filterleiste	Bedingungen als Chips («status = offen»), «+ Filter», Umschalter «WHERE frei»; Zeilenzahl rechts (genau oder «≈»)
Datenraster	virtualisiert; erste Spalte fixiert; Spaltenbreite ziehbar und gemerkt; waagrechtes Scrollen nur hier, mit Verlaufskante und Chevron (P-10)
Änderungsleiste	erscheint erst, wenn etwas geändert ist: «n Änderungen» · SQL ansehen · Verwerfen · Anwenden
SQL-Vorschau	Schublade von rechts oder unten; Mono; Hinweis je Treiber («in einer Transaktion» bzw. «nicht rückrollbar»); Kopieren · Im Editor öffnen
Bestätigung mit Namenseingabe	für DROP, TRUNCATE, DELETE ohne WHERE und jede Änderung auf Produktion: Zahl betroffener Zeilen, Name eintippen, Knopf erst dann aktiv
Detail-Schublade	alle Felder einer Zeile, lange Werte (JSON formatiert, Binärdaten mit Grösse und Herunterladen), Verweise vor und zurück — inline, nie ein Fenster in der Schublade
Fehlerkarte	Klartext oben («Die Spalte `total` darf nicht leer sein»), darunter der Originaltext des Servers mit Code, aufklappbar; nie gebundene Werte (maskiert)
Leerzustand	Symbol, ein Satz, nächster Schritt als Knopf («Zeile einfügen», «Daten importieren»)
Code-Eingabe	ein Feld für sechs Ziffern, `autocomplete="one-time-code"`, Einfügen aus der Mail; kein Zerlegen in sechs Kästchen
Befehlspalette	⌘K / Ctrl+K; Objekte, Aktionen, Verbindungen, zuletzt Benutztes; Pfeiltasten und Enter
Warnschild Konto	an der Verbindung: «Dieses Konto darf Dateien auf dem Server lesen» mit Link zur Empfehlung (V-10)

7. Text und Ton

- **Sie-Form**, kurze Sätze, Verben auf Knöpfen («Anwenden», «Verwerfen», «Verbindung testen»), keine Fachwörter, wo ein Alltagswort reicht — SQL-Begriffe bleiben im Original (`WHERE`, `NULL`).
- **Fehler** nennen Ursache und nächsten Schritt; der Servertext steht darunter, nicht stattdessen.
- **Englisch** britisch, gleiche Struktur; Schlüssel der Übersetzung = deutscher Text (wie Basalto).

8. Schmale Fenster

- Unter 768 px: Navigator als Schublade, Reiter brechen um, Raster wird zur **Kartenliste** (eine Karte je Zeile, die ersten vier Spalten, Rest in der Detail-Schublade). Bearbeiten bleibt möglich, ist aber auf den Desktop ausgelegt.
- **Keine Scrollbalken** (Regel Hoang, 15.10.2026): Seiten haben eine Verlaufskante mit Chevron, solange unten mehr folgt; waagrecht nur im Raster auf breiten Fenstern.

9. Bewegung

Kurz und funktional: 120–160 ms für Schubladen und Menüs, keine Animation im Raster; `prefers-reduced-motion` schaltet alles ab.

10. Entscheide (Annahme bis zur Sichtprüfung mit TB 00)

Nr.	Frage	Vorschlag
G-01	Grundfarben	Schiefergrau mit einem Akzent Petrol `#0b6e82` (dunkel `#4fc3d9`)
G-02	Schriften	Inter für die Oberfläche, JetBrains Mono für SQL und Bezeichner — beide OFL, im Build mitgeliefert
G-03	Umgebungsfarben	lokal Schiefer, Test Bernstein, Produktion Rot — mit Punkt-Symbol und Präfix im Seitentitel
G-04	Dichte	Standard 32 px Zeilenhöhe, «kompakt» 28 px je Konto wählbar
G-05	Dunkelmodus	gleichwertig, folgt dem System, umschaltbar im Kontomenü
G-06	Wortmarke	«phpdbadmin» in JetBrains Mono, «db» im Akzent; ein Zeichen folgt vor dem ersten öffentlichen Auftritt (als Pfade, nicht als Text)
G-07	Datenraster auf dem Telefon	Kartenliste statt Tabelle
G-08	Bestätigungen	Namenseingabe nur bei zerstörenden Handlungen und auf Produktion — sonst genügt die Vorschau

Anhang B — Wireframes — phpdbadmin

Stand: 28.10.2026 · v0.1 — Fragen F-1 bis F-6 von Hoang am 28.10.2026 entschieden (siehe unten), Raster auf Endlos-Scrollen umgestellt; die Sichtprüfung am echten Bildschirm folgt mit TB 00. Grundlage: Gestaltung (verbindlich), CR-001 § 3.3 und § 4, Produktprinzipien P-01 bis P-11, Befunde der Feasibility und die Skizze Bedienkonzept.

Zweck

Die Wireframes zeigen jeden Kernbildschirm von phpdbadmin auf dem Desktop (1440 × 900) und auf dem Telefon (375 × 812), in der Gestaltung der Foundation: Farben, Umgebungen, Rasterzustände, Typografie und Bausteine. Sie sind die Vorlage für die Umsetzung in Vue und die Abnahme je Bildschirm. Alle Daten sind erfunden (Kunden «Anna Muster», «Beat Keller» …, Team «Nina Brügger», «Reto Imhof», «Sandra Kälin»); echt ist nur das Konto von Hoang.

Übersicht zum Durchblättern: index.html — alle Bereiche, Desktop und Mobil nebeneinander, mit Links auf die HTML-Seiten.

Aufruf

```
cd dokumentationen/03-foundation/mockups
python3 mockup-bauen.py               # alles neu: Seiten, Bilder, index.html
python3 mockup-bauen.py --ohne-bilder # nur HTML (schnell, ohne Chrome)
python3 mockup-bauen.py --nur 05,06   # nur diese Bereiche
```

- **Voraussetzungen:** Python 3.11+ mit Pillow (nur für das Zuschneiden der mobilen Aufnahmen), Google Chrome unter `/Applications/Google Chrome.app`. Inter und JetBrains Mono werden genommen, wenn sie auf dem Rechner installiert sind; sonst greift die Ersatzschrift (`system-ui` bzw. `Menlo`).
- **Farben** liest der Generator aus `werkzeuge/kontraste.py` — der einzigen Quelle der Palette. Ändert sich dort eine Farbe, genügt ein neuer Lauf.
- **Seiten** (`seiten/*.html`) sind statisch: eingebettetes CSS, kein JavaScript, keine Netzwerkzugriffe. Jeder Selektor beginnt mit `.mk-`.
- **Aufnahmen** macht Chrome headless (vier gleichzeitig). Desktop direkt in 1440 × 900; Mobil in einem 500 px breiten Fenster (Chrome klemmt schmalere Fenster) mit festem 375-px-Rahmen, danach mit Pillow auf 375 × 812 zugeschnitten. Veraltete Bilder löscht ein voller Lauf.
- Dauer: meist unter einer Minute; im Hintergrund gedrosselt kann ein Lauf deutlich länger dauern.

Bereiche

37 Bilder: 14 × Desktop, 14 × Mobil und 9 Zusatzbilder (Zustände, Dunkelmodus, zweiter Treiber).

Nr.	Titel	Inhalt	Anforderungen	Bilder
01	Anmeldung	«Mit Passkey anmelden» als Hauptknopf, darunter E-Mail-Adresse und «Code senden»; Sprache wählbar, keine Registrierung, kein Passwort	A-01, A-02, A-03, A-06	Desktop · Mobil

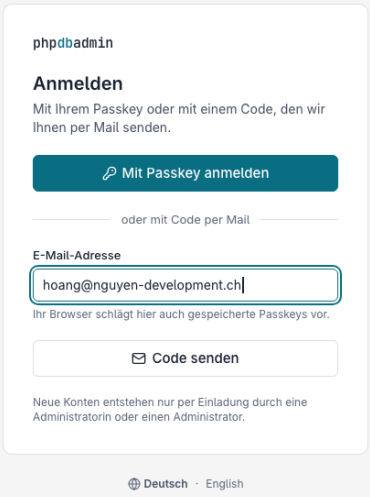
Nr.	Titel	Inhalt	Anforderungen	Bilder
02	Code eingeben	Ein Feld für sechs Ziffern, «Code erneut senden» mit Wartezeit; Fehlerzustand «Code abgelaufen» mit nächstem Schritt	A-02	Desktop · Mobil, abgelaufen · Desktop, abgelaufen
03	Startseite: Verbindungen	Statt Dashboard: Verbindungen in Gruppen, Umgebung mit Symbol und Wort, Treiber-Schild, Schloss bei Schreibschutz, Favoriten, Kontowarnung V-10 an «lokal · basalto», «Verbindung anlegen», zuletzt Geöffnetes	A-04, V-04, V-06, V-10	Desktop · Mobil
04	Verbindung anlegen	Treiber als vier Kacheln, Felder je Treiber, Umgebung, Schreibschutz, «Passwort nicht speichern», Test in Schritten. Desktop: MySQL auf Produktion mit «Zugang verweigert (1045)»; Mobil: SQLite-Datei im erlaubten Ordner, Test erfolgreich mit Version	V-01 bis V-05	Desktop · Mobil
05	Arbeitsbildschirm Daten	Nach der Skizze: Produktions-Chip und roter Streifen, Schreibfreigabe auf Zeit, Befehlspalette-Feld, Navigator, Reiter mit Umgebung, Filter-Chips, Raster mit geänderten Zellen, neuer Zeile, Zeile zum Löschen, NULL, Fremdschlüssel-Pfeil, Änderungsleiste, «Rückgängig» der letzten Anwendung	O-01 bis O-04, D-01 bis D-04, D-06, D-09, V-04	Desktop · Mobil · Desktop dunkel · Mobil dunkel · Mobil, Navigator als Schublade
06	SQL-Vorschau vor dem Anwenden	Schublade mit jeder Anweisung, gezählten Zeilen und Folgen (ON DELETE CASCADE), Hinweis je Treiber (MySQL/InnoDB: in einer Transaktion), Bestätigung mit Namenseingabe auf Produktion — Desktop halb getippt (Knopf aus), Mobil fertig (Knopf aktiv)	D-04, D-09, Q-07, N-03	Desktop · Mobil
07	Zeilen-Detail-Schublade	Alle Felder mit Typ, geänderter Wert mit Vorher-Wert, JSON formatiert und geprüft, NULL gegen leer, Binärwert mit Grösse und Herunterladen, Zeitzone, Verweise vor und zurück, «Zeile kopieren als»	D-03, D-05, E-05	Desktop · Mobil
08	Struktur einer Tabelle	Spalten mit Originaltyp und gemeinsamer Art, Kollation, Kommentar; Indizes, Fremdschlüssel samt eingehender Verweise, Check-Constraints, Trigger, DDL zum Kopieren; «Tabelle ändern» auf schreibgeschützter Produktion gesperrt	S-01, O-04, V-04	Desktop · Mobil
09	Tabelle ändern mit Vorschau	Formular mit neuer, umbenannter und gelöschter Spalte und neuem Index, daneben das erzeugte SQL. MySQL: «Nicht rückrollbar — Anweisung für Anweisung»; Spalte löschen verlangt den Tabellennamen. Zusatzbild PostgreSQL: gleiche Oberfläche, «in einer Transaktion»	S-03, S-04, N-03, O-04	Desktop (MySQL, Test) · Mobil · Desktop PostgreSQL
10	SQL-Editor	Hervorhebung, Vervollständigung aus dem Schema, Ausführen (aktuelle Anweisung, Auswahl, alles), Ergebnisse als Reiter mit Zeilen und Dauer, Zustand «abgebrochen nach 30 s» (MySQL meldet ihn nicht selbst), Gespeichertes und Verlauf	Q-01 bis Q-04, Q-09, O-05	Desktop · Mobil, abgebrochen · Desktop dunkel, abgebrochen

Nr.	Titel	Inhalt	Anforderungen	Bilder
11	Export und Import	CSV-Import in `customers`: Spaltenzuordnung, Probelauf (nichts geschrieben, drei Fehlerzeilen in Klartext), Statistik danach aktualisieren. Export-Dialog CSV/JSON/SQL mit Aufträgen im Hintergrund und Fortschritt	E-01, E-02, E-06, B-07	Desktop, Import · Mobil, Export · Desktop, Export
12	Befehlspalette ⌘K	Über dem Arbeitsbildschirm: Treffer nach aktueller Verbindung, anderen Verbindungen (mit Umgebung) und Aktionen; Tastatur-Hinweise. Mobil als ganze Seite	O-02, O-05	Desktop · Mobil
13	Konto	Passkeys mit Name, Datum und letzter Nutzung, Sitzungen mit «Beenden», Farbschema, Dichte, Sprache; Name und E-Mail nur lesend — keine Profilseite. Zusatzbild: Kontomenü	A-03, A-05, A-08, O-06	Desktop · Mobil · Desktop, Kontomenü
14	Verwaltung	Konten und Einladungen, Rechte je Verbindung (kein Zugriff, lesen, schreiben, verwalten), persönliche Verbindungen nicht geteilt. Zusatzbild: Protokoll aller schreibenden Aktionen mit Platzhaltern statt Werten und Aufbewahrungsfrist	A-06, A-07, A-09, V-09, B-04	Desktop · Mobil · Desktop, Protokoll

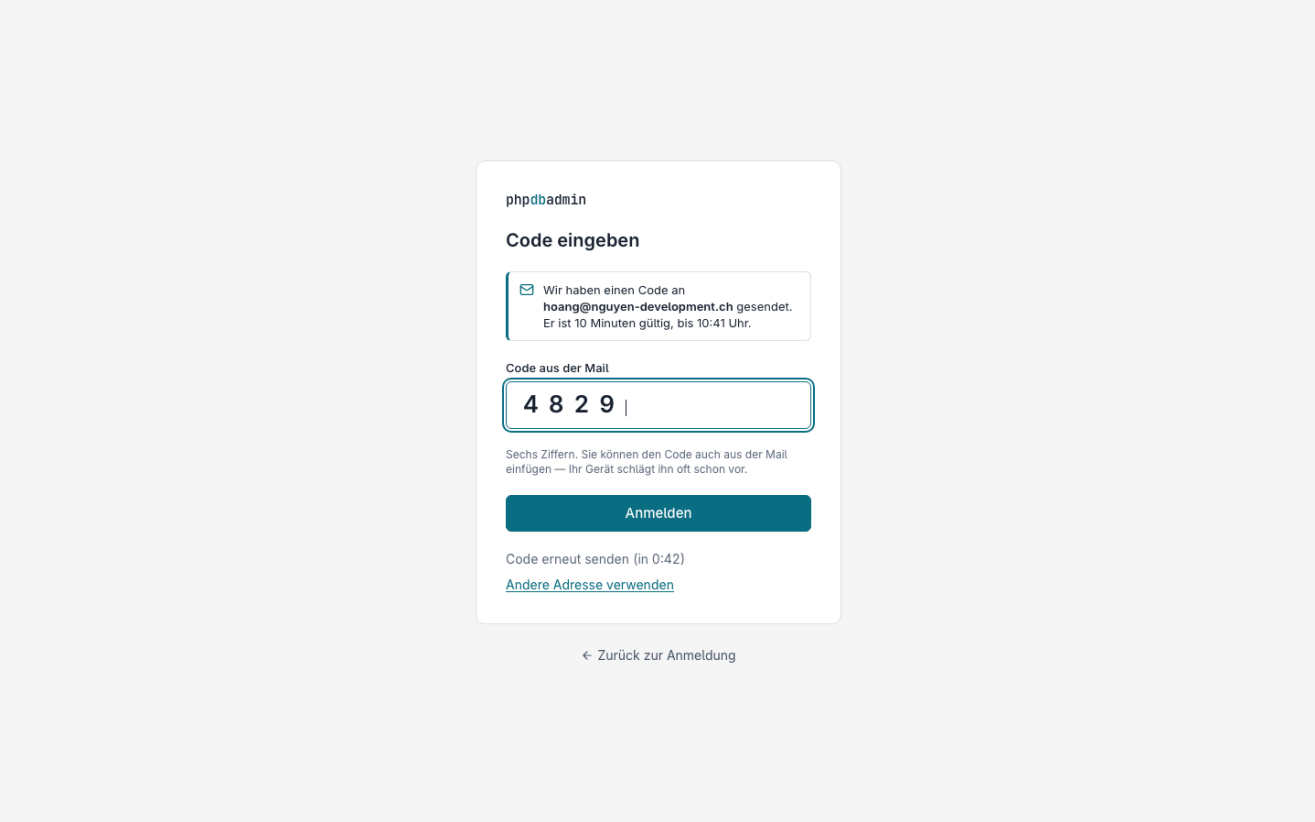
Bildstrecke (Desktop)

Die 14 Desktop-Bilder in Originalgrösse 1440 × 900; mobile Fassungen und Zusatzbilder sind in der Tabelle oben verlinkt.

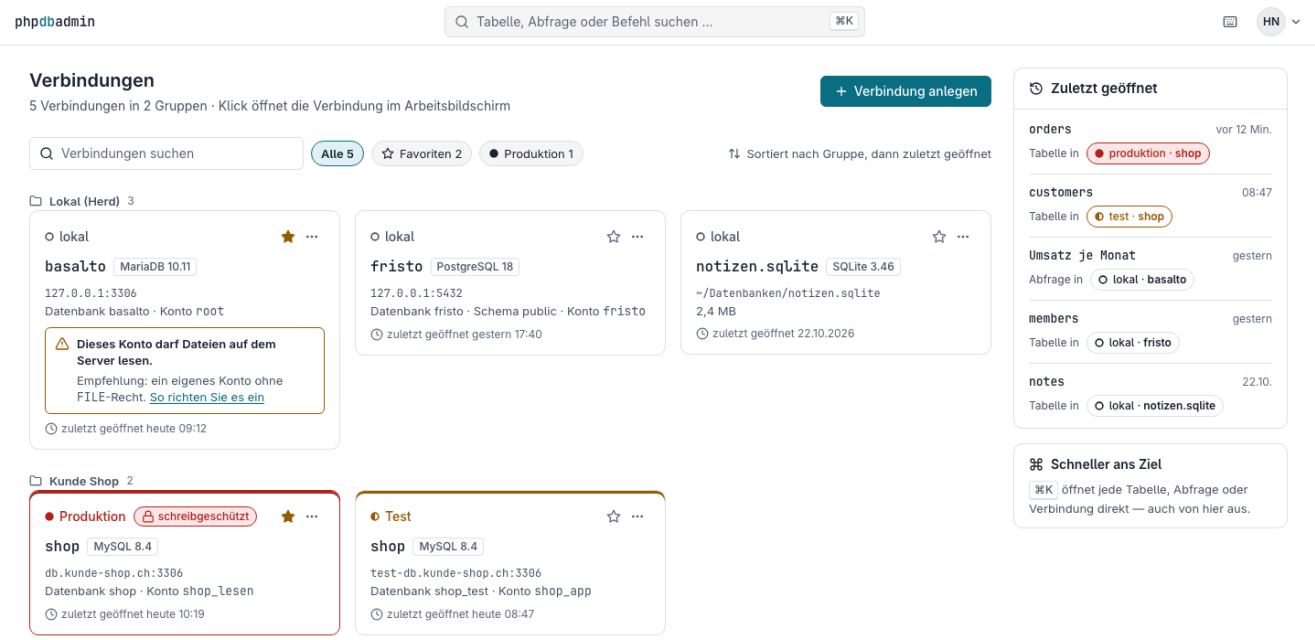
01 — Anmeldung



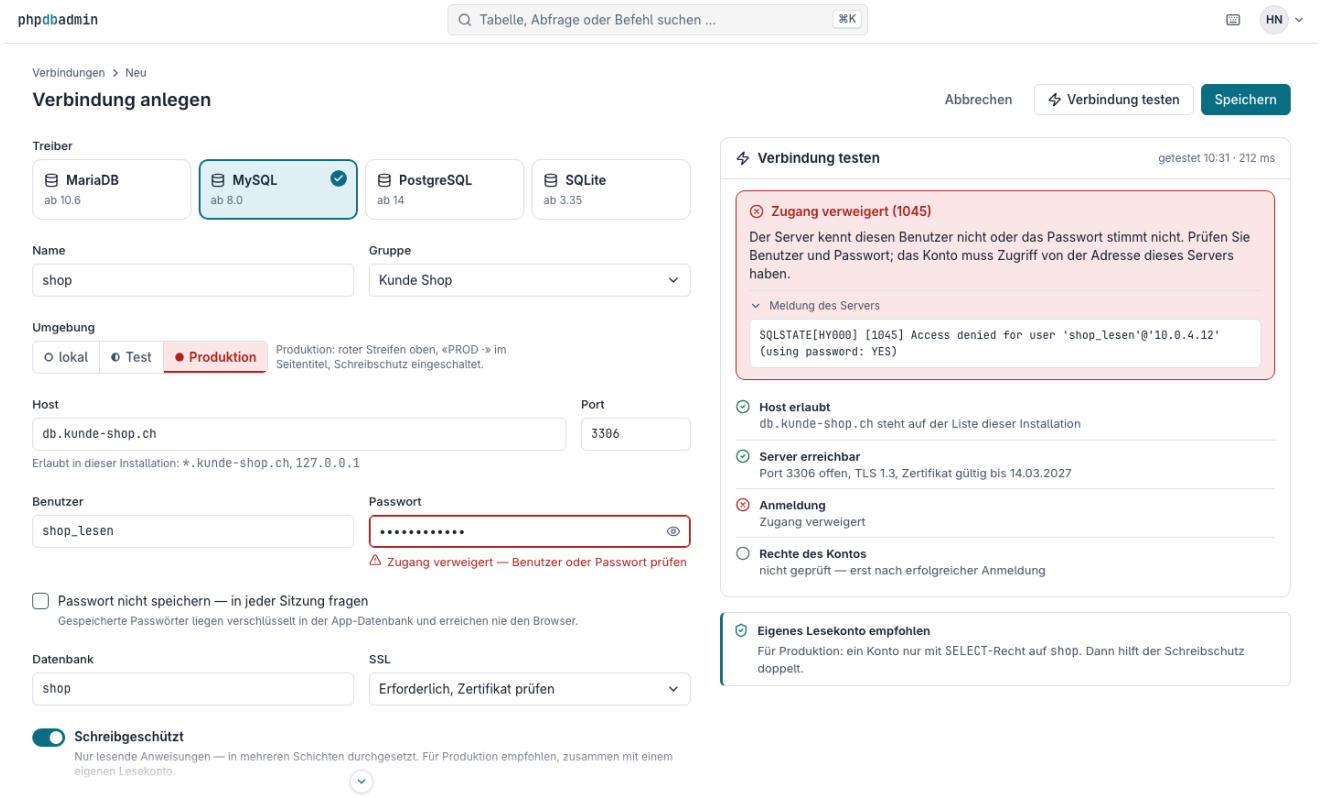
02 — Code eingeben



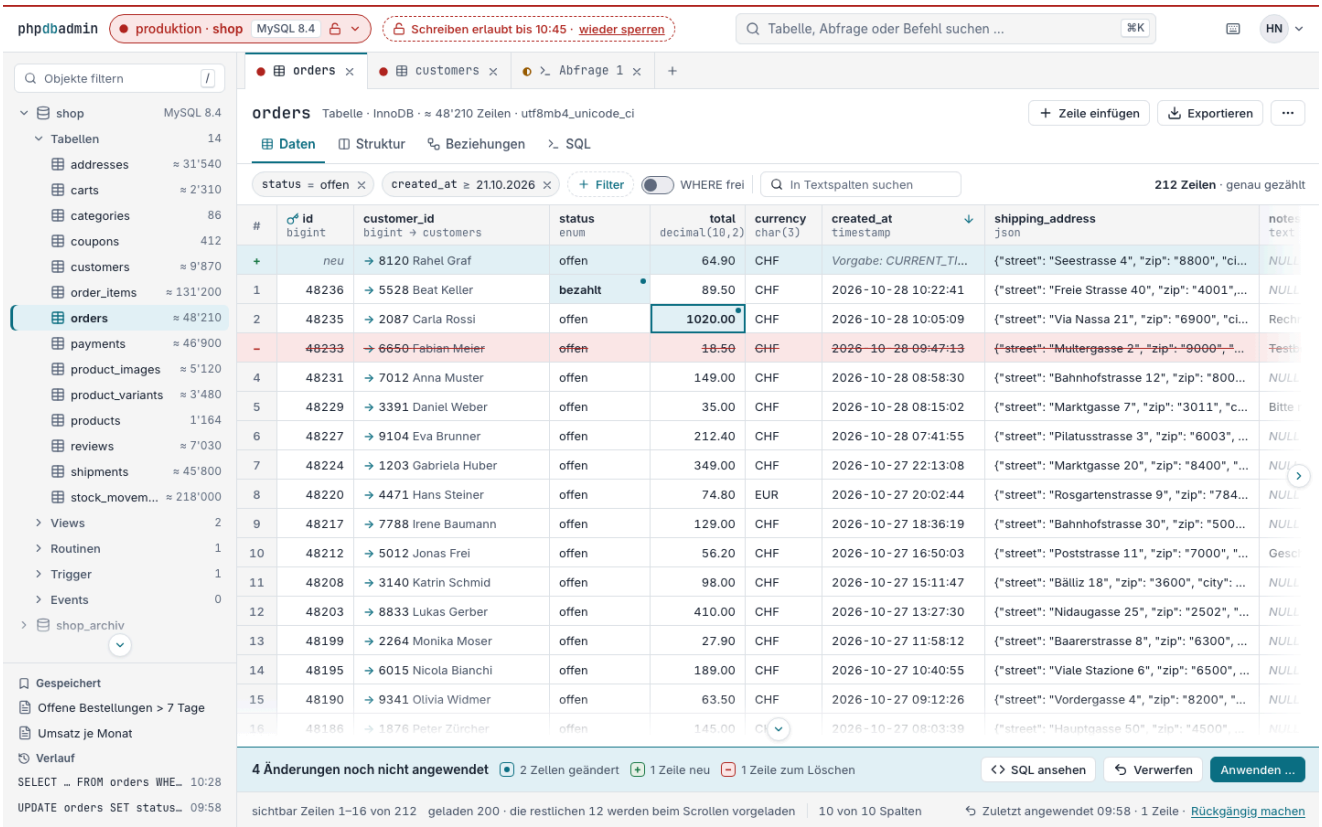
03 — Startseite: Verbindungen



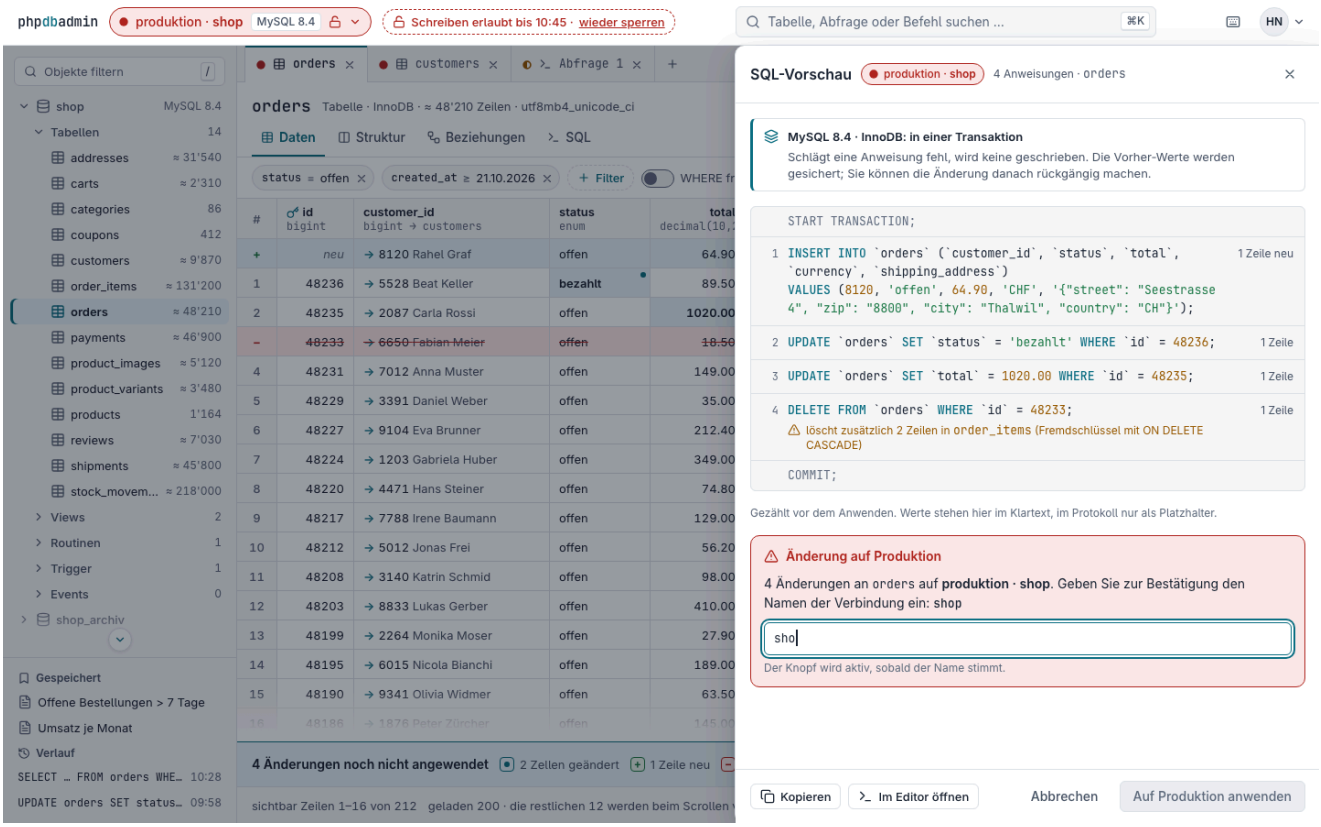
04 — Verbindung anlegen



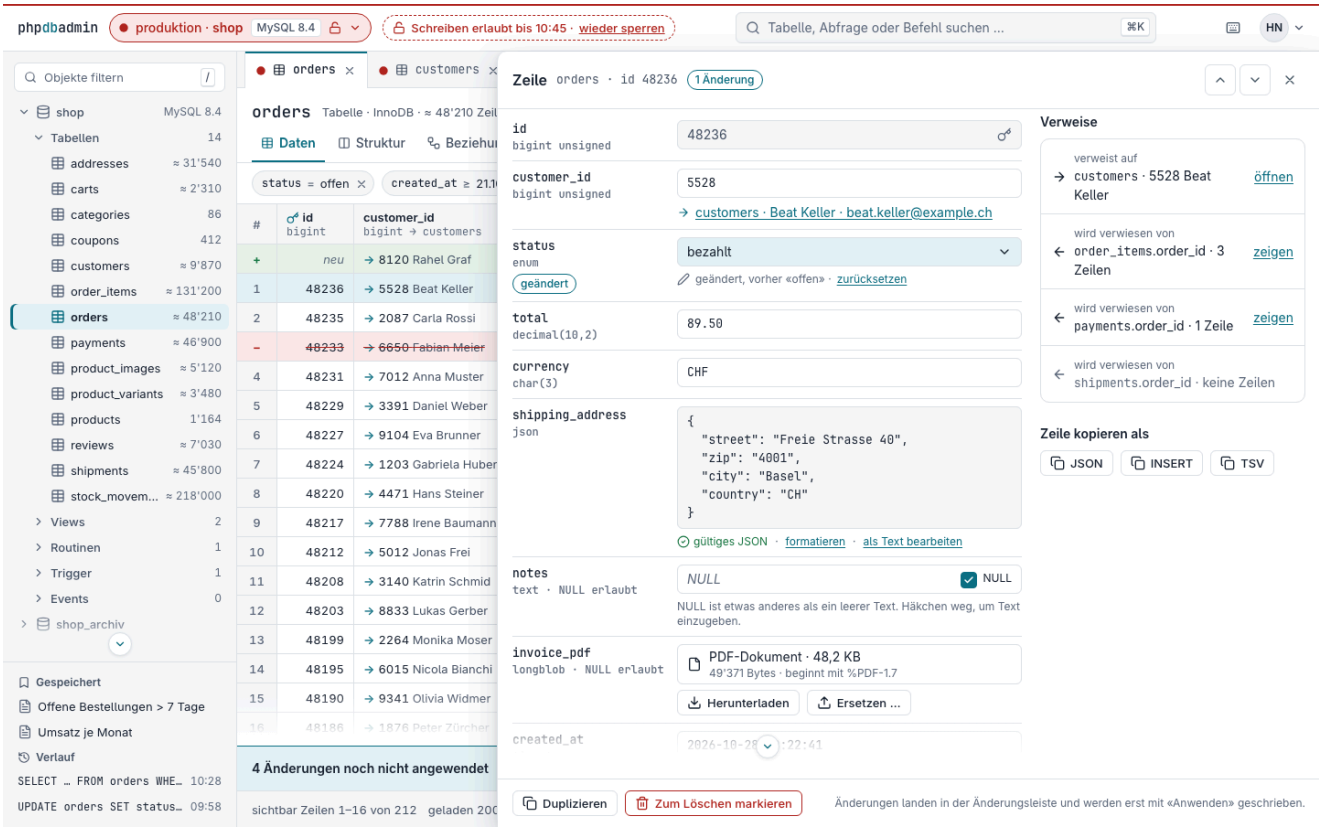
05 — Arbeitsbildschirm Daten



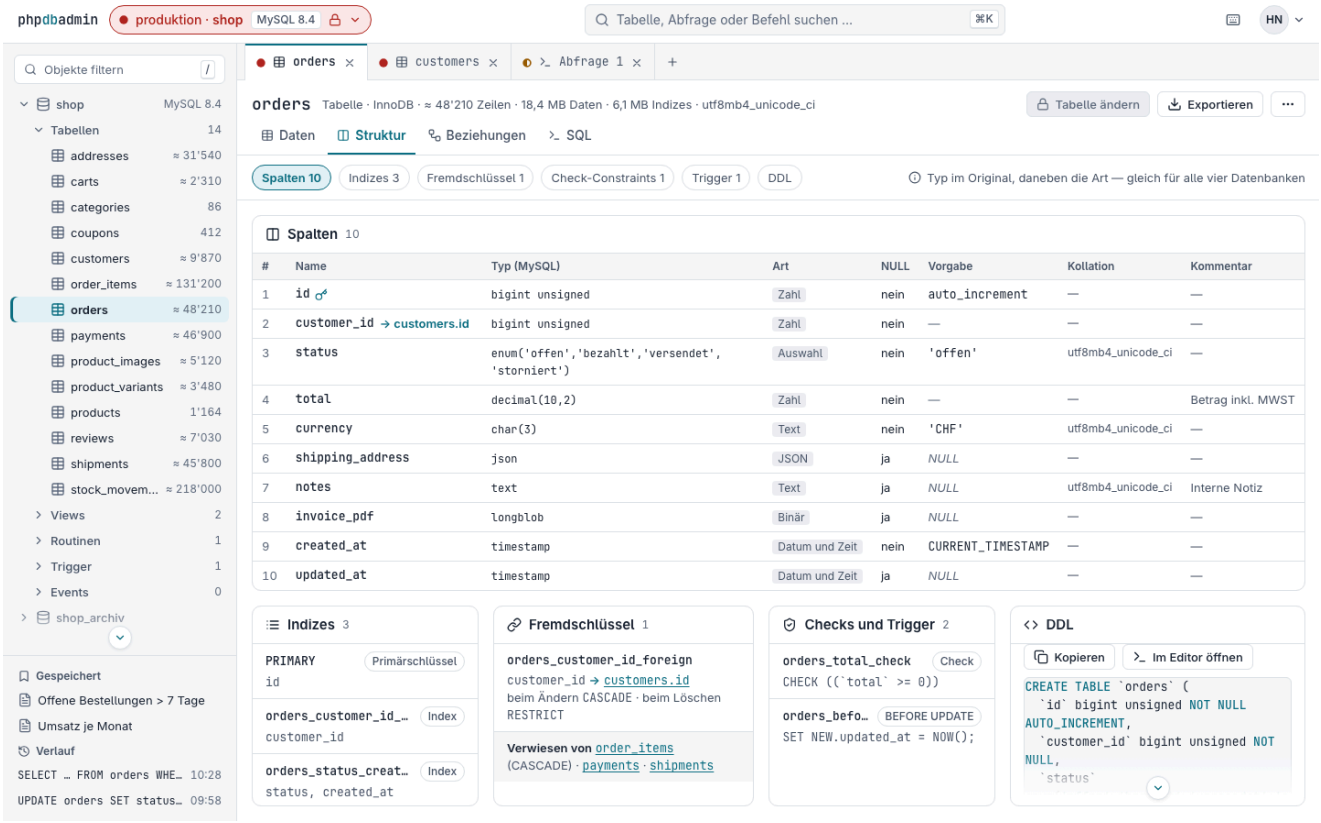
06 — SQL-Vorschau vor dem Anwenden



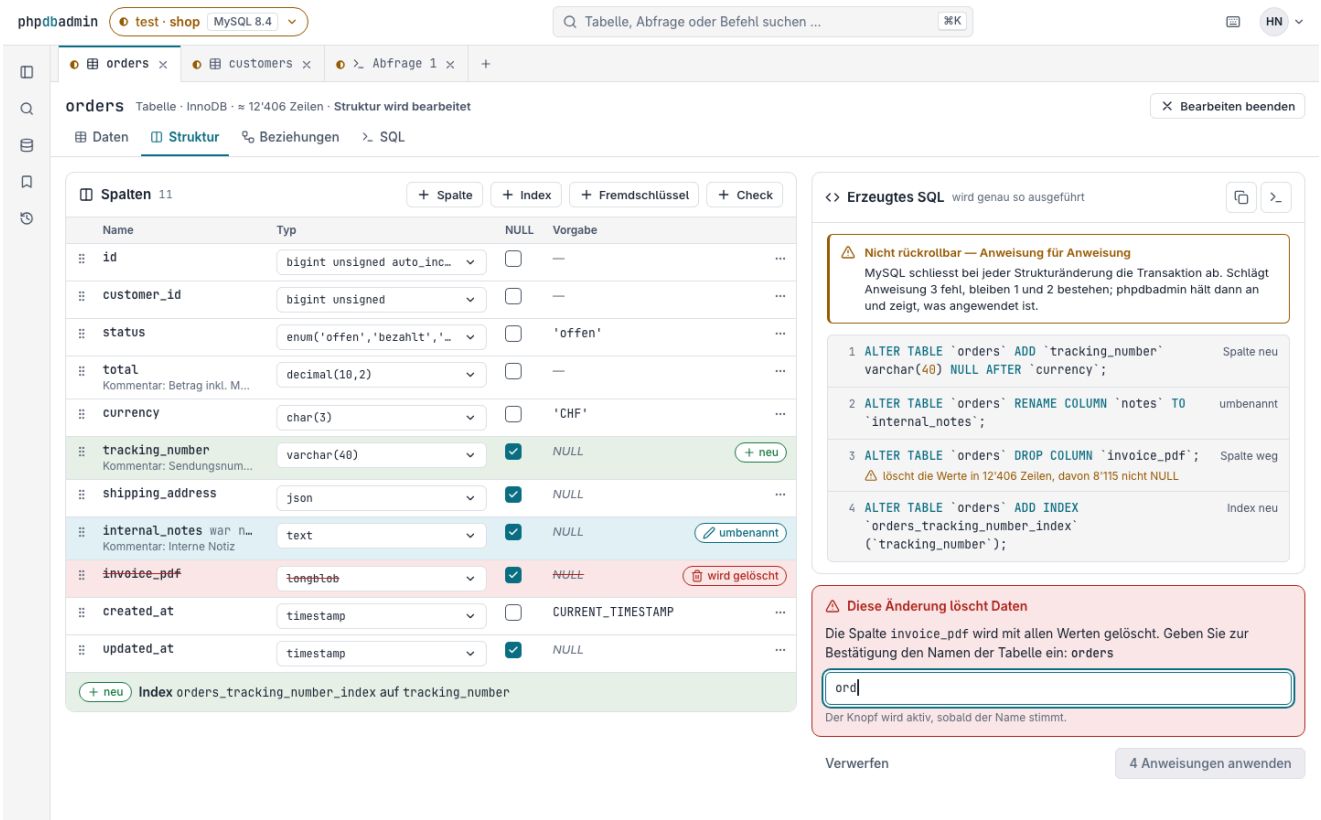
07 — Zeilen-Detail-Schublade



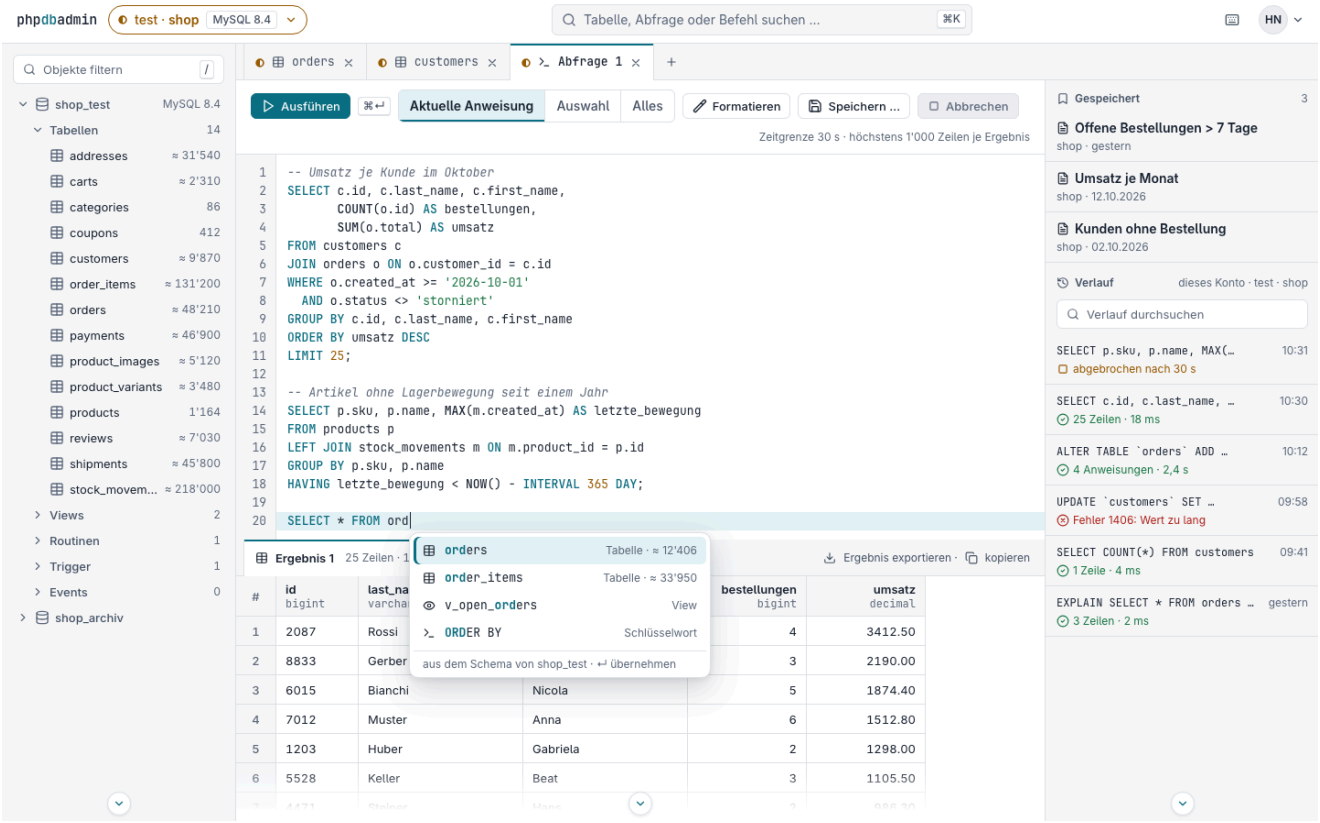
08 — Struktur einer Tabelle



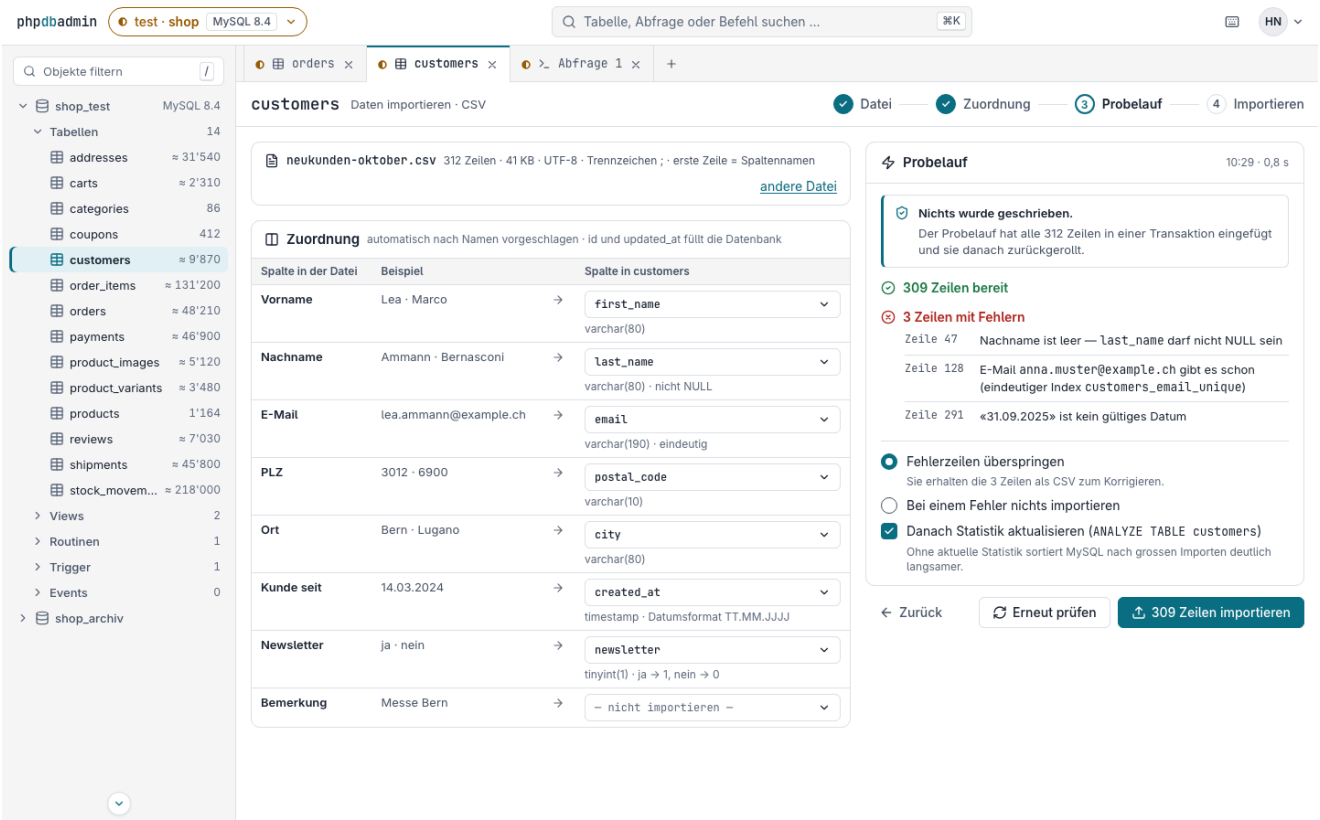
09 — Tabelle ändern mit Vorschau



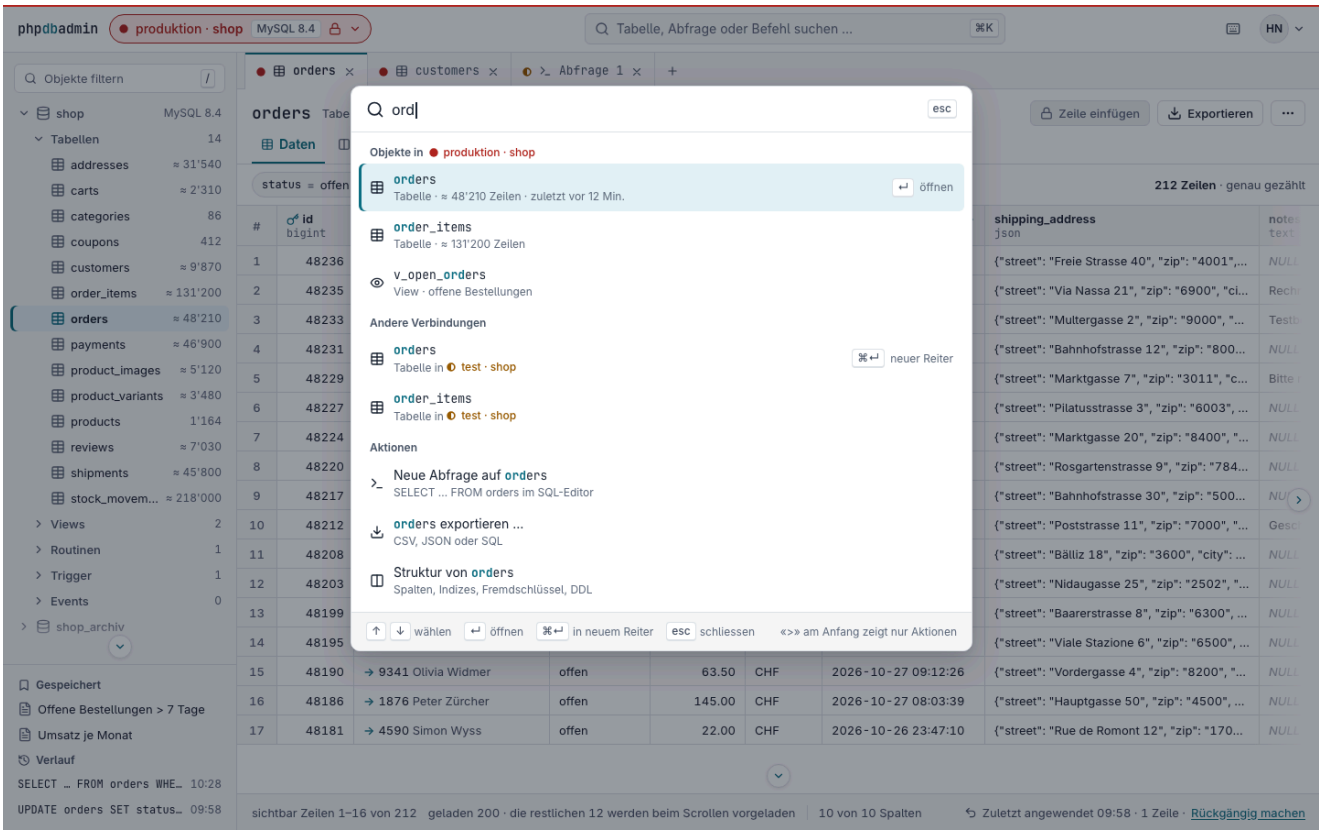
10 — SQL-Editor



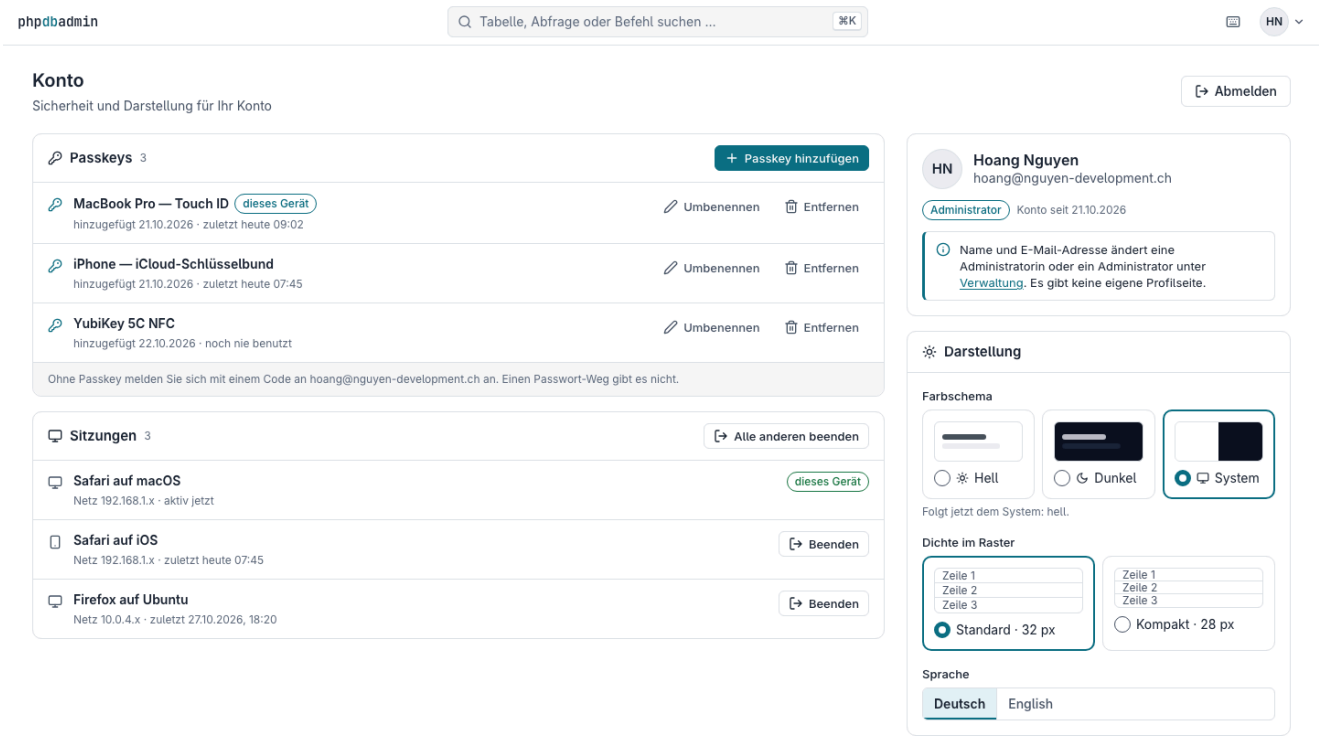
11 — Export und Import



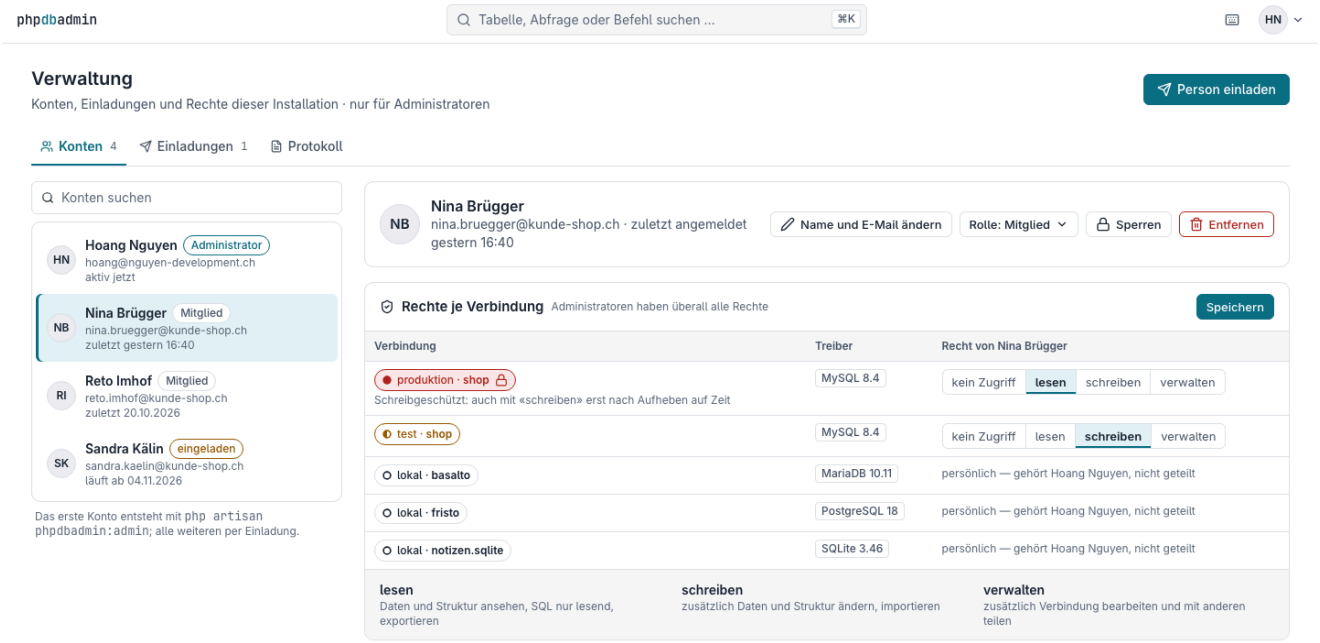
12 — Befehlspalette ⌘K



13 — Konto



14 — Verwaltung



Entscheide

Gestaltende Entscheide dieser Wireframes — gelten als Annahme bis zur Sichtprüfung mit TB 00, wie G-01 bis G-08 der Gestaltung; die Fragen F-1 bis F-6 sind entschieden.

Nr.	Entscheid	Begründung
E-01	**Passkey ist der Hauptknopf** der Anmeldung; der Code per Mail ist der zweite Weg darunter. Das E-Mail-Feld schlägt zugleich Passkeys vor (`autocomplete="username webauthn"`).	Im Alltag ist der Passkey ein Klick ohne Eingabe (Skizze Anmeldung, Weg 2); der Code ist für neue Geräte. Ein Akzent je Bereich (gestaltung § 2).
E-02	**Code erneut senden** ist mit sichtbarer Wartezeit gesperrt («in 0:42»); bei «abgelaufen» bleibt der Code im Feld stehen, die Fehlerkarte nennt Gültigkeit und nächsten Schritt.	A-02 drosselt; eine Sperre ohne Zeitangabe wirkt wie ein Fehler. Fehler nennen Ursache und nächsten Schritt (§ 7).
E-03	**Startseite als Karten in Gruppen**, Favoriten als Stern und Filter (keine eigene Favoritengruppe), rechts «Zuletzt geöffnet» statt Kennzahlen.	Kein Dashboard (A-04); eine Verbindung soll nur einmal erscheinen. «Zuletzt geöffnet» ist der kürzeste Weg zurück an die Arbeit.
E-04	**Umgebung immer als Punkt und Wort**, Name im Muster «umgebung · name»; Produktion zusätzlich mit rotem Streifen und rotem Kartenrahmen, Test mit bernsteinfarbener Oberkante; die gewählte Umgebung im Formular trägt ihre eigene Farbe, nicht den Akzent.	Farbe nie allein (WCAG 1.4.1, § 2); die Umgebung muss überall gleich lesbar sein — im Chip, in Reitern, Listen und der Palette.

Nr.	Entscheid	Begründung
E-05	**Schreibschutz auf Zeit aufheben:** «produktion · shop» ist schreibgeschützt (Bereich 3, 8, 11, 12). Für die Bereiche 5–7 hat Hoang ihn bis 10:45 aufgehoben — sichtbar als gestrichelter Chip «Schreiben erlaubt bis 10:45 · wieder sperren» und offenes Schloss; das Protokoll hält es mit Grund fest.	Sonst widersprechen sich «Produktion standardmässig schreibgeschützt» (V-04) und Bearbeiten auf Produktion (Skizze, Bereiche 5 und 6). Siehe Frage F-1.
E-06	**«Auf Produktion anwenden» ist ein Gefahr-Knopf** und erst nach der Namenseingabe aktiv; auf dem Telefon steht die Namenseingabe angeheftet direkt über dem Knopf. Dieselbe Bestätigung mit dem Tabellennamen gilt für zerstörende Strukturänderungen auch ausserhalb von Produktion.	G-08; Gefahr ist für Produktion erlaubt (§ 2). Ohne Anheften wäre auf dem Telefon ein aktiver roter Knopf ohne sichtbare Bestätigung zu sehen.
E-07	**SQL-Vorschau zählt je Anweisung** die betroffenen Zeilen und nennt Folgen über Fremdschlüssel («löscht zusätzlich 2 Zeilen in `order_items`»); Werte stehen in der Vorschau im Klartext, im Protokoll nur als `?`.	P-02 und P-04: nichts passiert ungesehen, betroffene Zeilen vorher zählen. B-04 verlangt maskierte Werte im Protokoll.
E-08	**Werte im Raster roh** wie gespeichert (`1020.00`, `2026-10-28 10:22:41`), Zähler der Oberfläche mit Schweizer Apostroph (48'210) und «≈» bei Schätzungen.	Rohwerte lassen sich unverändert kopieren und bearbeiten; die Skizze zeigte «1 020.00» nur als Skizze. Geschätzt und genau müssen unterscheidbar sein (D-06).
E-09	**Fremdschlüssel zeigt Schlüssel und Anzeigewert** des Ziels («→ 5528 Beat Keller»).	Eine nackte Zahl sagt nichts; der Pfeil bleibt der Klick zum Verweis (§ 3). Kostet eine Abfrage je Seite — siehe F-5.
E-10	**Neue Zeilen stehen oben** im Raster, Zeilen zum Löschen bleiben bis «Anwenden» sichtbar; die Legende der Änderungsleiste trägt Zeichen (Punkt, +, –).	Was gesammelt ist, soll ohne Suchen sichtbar sein. Die Legende war zuerst nur Farbe auf gleich farbiger Leiste — unsichtbar.
E-11	**Detail-Schublade breit** (900 px, zwei Spalten: Felder links, Verweise und «Zeile kopieren als» rechts); Zeiten in Europe/Zurich mit dem gespeicherten UTC-Wert darunter; NULL als eigenes Kästchen neben dem Feld.	Nur so passen JSON, Binärwert und Verweise ohne Fenster in der Schublade (§ 6, D-05). NULL und leer müssen unterscheidbar bleiben (§ 3).
E-12	**Struktur zeigt den Typ im Original und eine gemeinsame Art** (Zahl, Text, JSON, Binär, Datum und Zeit, Auswahl); «Tabelle ändern» bleibt bei Schreibschutz sichtbar, aber gesperrt mit Schloss.	Feasibility F-1 und O-04: gleiche Begriffe für vier Datenbanken. Eine versteckte Funktion erklärt nicht, warum sie fehlt.
E-13	**Tabelle ändern** klappt den Navigator zur Schiene ein und zeigt das erzeugte SQL laufend neben dem Formular, mit Hinweis je Treiber über den Anweisungen.	Formular und Vorschau brauchen zusammen die Breite; das SQL soll nie ein zusätzlicher Schritt sein (P-02, S-03).
E-14	**Hervorhebung im Editor** mit Akzent (Schlüsselwörter), Erfolg (Zeichenketten), Warnung (Zahlen) und leise kursiv (Kommentare), nie Rot; **Ligaturen in JetBrains Mono aus**.	Rot ist Gefahr vorbehalten (§ 1). Mit Ligaturen erschienen `>=` als ≥ und `<>` als Raute — im SQL muss stehen, was getippt ist.
E-15	**«Abgebrochen» ist eine Warnung** (Bernstein), kein Fehler, mit gemessener Zeit, `KILL QUERY` und drei nächsten Schritten.	Feasibility F-2: MySQL kehrt still zurück, die Anwendung muss es selbst sagen; ein Zeitlimit ist kein Fehler des Nutzers.
E-16	**Import in vier Schritten** mit dem Probelauf als eigenem Schritt (in einer Transaktion, danach zurückgerollt); «Statistik aktualisieren» ist vorgewählt.	E-02 verlangt «Probelauf schreibt nichts»; E-06 aus der Feasibility (Sortieren ohne Statistik 100-mal langsamer).
E-17	**Aufträge im Hintergrund** als schwebende Karte unten rechts (Telefon: oben im Blatt) mit Fortschritt, Abbrechen und Download bis zum Folgetag.	E-01: grosse Exporte im Hintergrund; Arbeiten soll weitergehen.

Nr.	Entscheid	Begründung
E-18	**Befehlspalette** gruppiert nach aktueller Verbindung, anderen Verbindungen (mit Umgebung) und Aktionen; «>» am Anfang zeigt nur Aktionen.	O-02; eine Tabelle gleichen Namens auf Test und Produktion muss in der Palette unterscheidbar sein.
E-19	**Konto ist eine Seite** mit Sicherheit und Darstellung; das Kontomenü führt zu Sicherheit, Darstellung, Verwaltung (nur Administratoren) und Abmelden; Name und E-Mail stehen nur lesend mit Hinweis auf die Verwaltung.	A-05: keine Profilseite. Die Farbschema-Wahl «System» ist Vorgabe (G-05).
E-20	**Rechte als vierstufiger Umschalter** je Verbindung (kein Zugriff, lesen, schreiben, verwalten) mit Erklärung darunter; persönliche Verbindungen erscheinen, sind aber nicht teilbar.	A-07, V-09. «Schreiben» auf einer schreibgeschützten Verbindung wirkt erst nach Aufheben auf Zeit — das steht direkt an der Zeile.
E-21	**Mobil:** Raster als Kartenliste (id im Kopf, dann `customer_id`, `status`, `total`), Änderungsleiste unten fest, Navigator als Schublade von links mit Verbindungswechsel oben, Vorschau und Export als Blatt von unten, Palette als ganze Seite.	Gestaltung § 8 und G-07; Bearbeiten bleibt möglich, Struktur ändern ist ausdrücklich «für den Desktop ausgelegt».
E-22	**Fett ist 600**, nur die Zeilenzeichen «+» und «–» und Treffer in der Palette sind 700. Die Palette kommt ausschliesslich aus `kontraste.py`; abgeleitet sind nur Schleier und Schatten.	Ruhige Oberfläche (§ 1); eine Quelle der Farben, damit die gerechneten Kontraste gelten.

Fragen an Hoang — entschieden 28.10.2026

Alle sechs wie empfohlen entschieden (Hoang, interaktiv: F-1 als neue Anforderung **V-11**, F-2 bis F-6 «wie empfohlen»). Dazu entschied Hoang, dass das Raster **endlos scrollt** (ADR-006) — die Fusszeile des Arbeitsbildschirms zeigt seither nur die Position («sichtbar Zeilen 1–16 von 212 · geladen 200»), keine Seitenknöpfe mehr.

Nr.	Frage	Empfehlung = Entscheid
F-1	Wie schreibt man auf eine schreibgeschützte Produktion? (a) Schreibschutz **auf Zeit aufheben** wie gezeigt (E-05), (b) nur über eine zweite, nicht geschützte Verbindung, (c) gar nicht.	**(a)** mit 15 Minuten, Pflichtgrund, nur mit Recht «schreiben», im Protokoll — als neue Anforderung V-11 in CR-001 aufnehmen.
F-2	Soll «Anwenden» auf Produktion **rot** (Gefahr) sein oder im Akzent bleiben wie überall sonst?	**Rot** — die Gestaltung erlaubt Gefahr für die Umgebung Produktion, und die Namenseingabe davor macht den Schritt bewusst.
F-3	Detail-Schublade **breit (900 px)** mit allen Feldern und Verweisen, oder **schmal (560 px)** mit langen Werten weiter unten?	**Breit**, mit ziehbarer und gemerkter Breite; auf dem Telefon ohnehin ganze Seite.
F-4	**Neue Zeilen oben** angeheftet bis «Anwenden», oder an der Stelle, an der sie nach Sortierung stehen würden?	**Oben** — sonst verschwindet eine neue Zeile bei grossen Tabellen auf einer anderen Seite.
F-5	**Anzeigewert am Fremdschlüssel** («⟶ 5528 Beat Keller») — eine zusätzliche Abfrage je Seite. Immer, abschaltbar, oder nie?	**Standardmässig an** (erste Textspalte des Ziels, nur für die sichtbare Seite), je Tabelle abschaltbar.
F-6	Soll die Startseite die **Verbindungsübersicht** bleiben oder direkt die **zuletzt geöffnete Verbindung** zeigen?	**Übersicht** mit «Zuletzt geöffnet» rechts (wie gezeigt); eine Einstellung «Startseite» frühestens in Stufe 4.

Grenzen der Wireframes

- **Nicht gezeichnet:** Ansicht «Beziehungen» (S-07, Stufe 3), Reiter «Einladungen» und der Einladungs-Dialog, Leer- und Ladezustände des Rasters (O-09; gezeigt sind nur Fehler- und Abbruchzustände), Import auf dem Telefon, Protokoll auf dem Telefon, Tabelle anlegen (S-02 nutzt das Formular aus Bereich 9).
- **Symbole** sind Lucide-Formen, von Hand als SVG-Pfade nachgezeichnet; einzelne weichen leicht vom Original ab.
- **Zusatzbild 09 PostgreSQL** nimmt eine angenommene Kopie «lokal · shop» auf PostgreSQL, damit dieselbe Tabelle auf zwei Treibern nebeneinander steht (O-04); sie gehört nicht zu den fünf Beispielverbindungen.
- Statische Seiten: Scrollen, Hover und Tastatur sind als Zustand gezeichnet (Verlaufskante mit Chevron, Fokusrahmen, Cursor), nicht bedienbar.

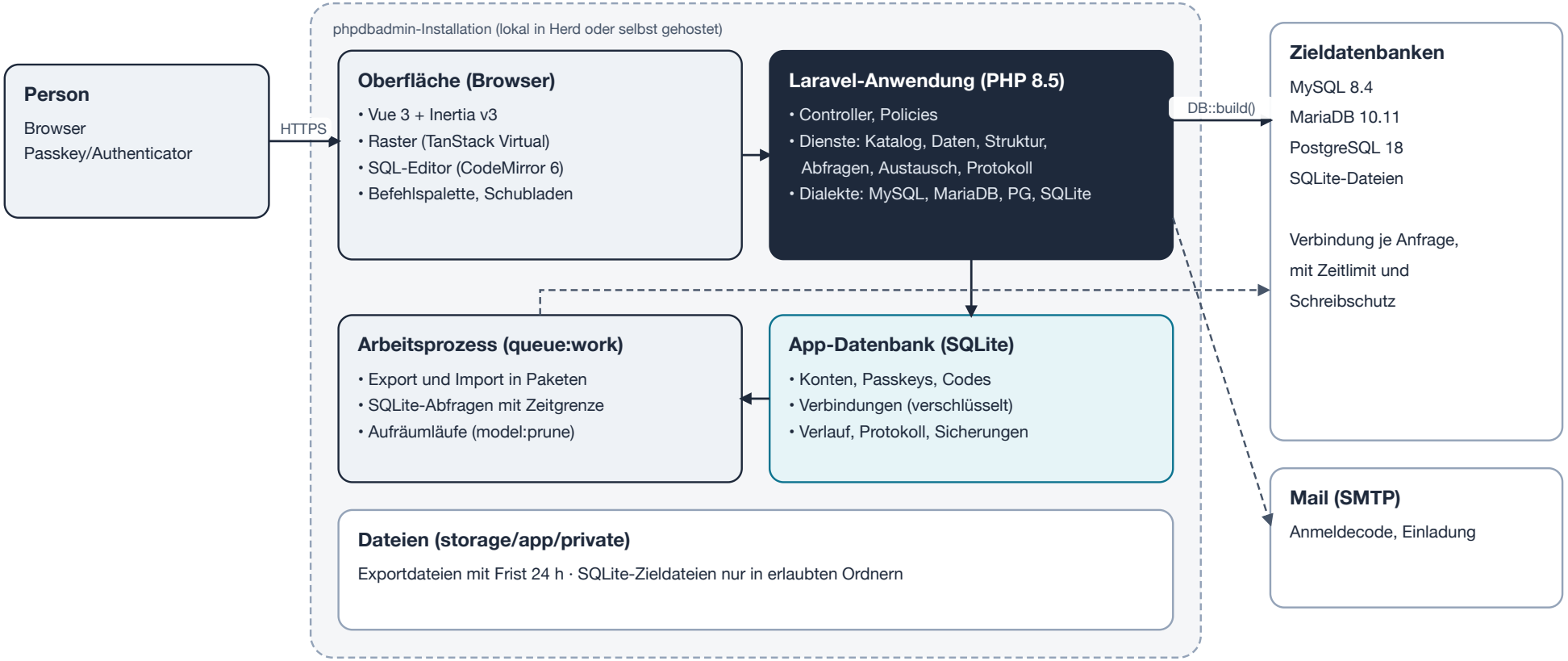
Anhang C — Solution-Architektur — phpdbadmin

Stand: 28.10.2026 · v1.0 (Foundation). Eingang: CR-001 Fassung 3, Feasibility v1.0, Terms of Reference v1.0. Die Entscheide A-01 bis A-14 in § 15 sind als ADR-001 bis ADR-011 ausformuliert (04-evolutionary-development/adr).

1. Überblick

phpdbadmin — Container (C4 Level 2)

Eine Laravel-Anwendung mit Arbeitsprozess und eigener App-Datenbank; Zieldatenbanken sind Umsysteme



Querschnitt hinter den Controllern: Recht vor Routenbindung (404 statt 403) · Grenzen · Maskierung von Meldungen · Protokoll schreibender Aktionen.

Kein Container für Suche, Cache-Server oder Push: Cache und Warteschlange laufen über die App-Datenbank.

phpdbadmin ist **eine Laravel-Anwendung** mit einer Oberfläche aus Inertia-Seiten, einem Arbeitsprozess für die Warteschlange und einer eigenen App-Datenbank. Die verwalteten Datenbanken sind Umsysteme: Die Anwendung baut je Anfrage eine Verbindung aus den gespeicherten Angaben, setzt Grenzen und gibt sie nach der Anfrage wieder frei. Was alle vier Treiber gleich können, macht Laravel; was sie unterscheidet, macht die **Dialekt-Schicht** (ADR-001).

2. Was die Plattform vorgibt

Vorgabe	Folge
PHP 8.5	`Pdo\Sqlite::setAuthorizer()` gibt es erst ab 8.5 (geprüft: 8.4.27 nein, 8.5.12 ja) — Mindestversion des Produkts (ADR-011); `composer.json` von `^8.3` auf `^8.5`
Laravel 13.39, Inertia v3, Vue 3, Tailwind 4, Wayfinder	wie installiert
Betrieb lokal (Herd) und selbst gehostet (R-11)	keine Mandanten, keine SaaS-Grenzen; eine Installation = eine App-Datenbank
Zieldatenbanken liegen oft in internen Netzen	phpdbadmin läuft neben ihnen; erlaubte Hosts sind Pflicht (V-05)
PHP-Anfragen sind synchron, PHP-FPM bearbeitet mehrere gleichzeitig	Abbrechen läuft als zweite Anfrage; Sitzungen ohne Sperre (Datenbank-Sitzungstreiber)

3. Schichten und Verzeichnisse

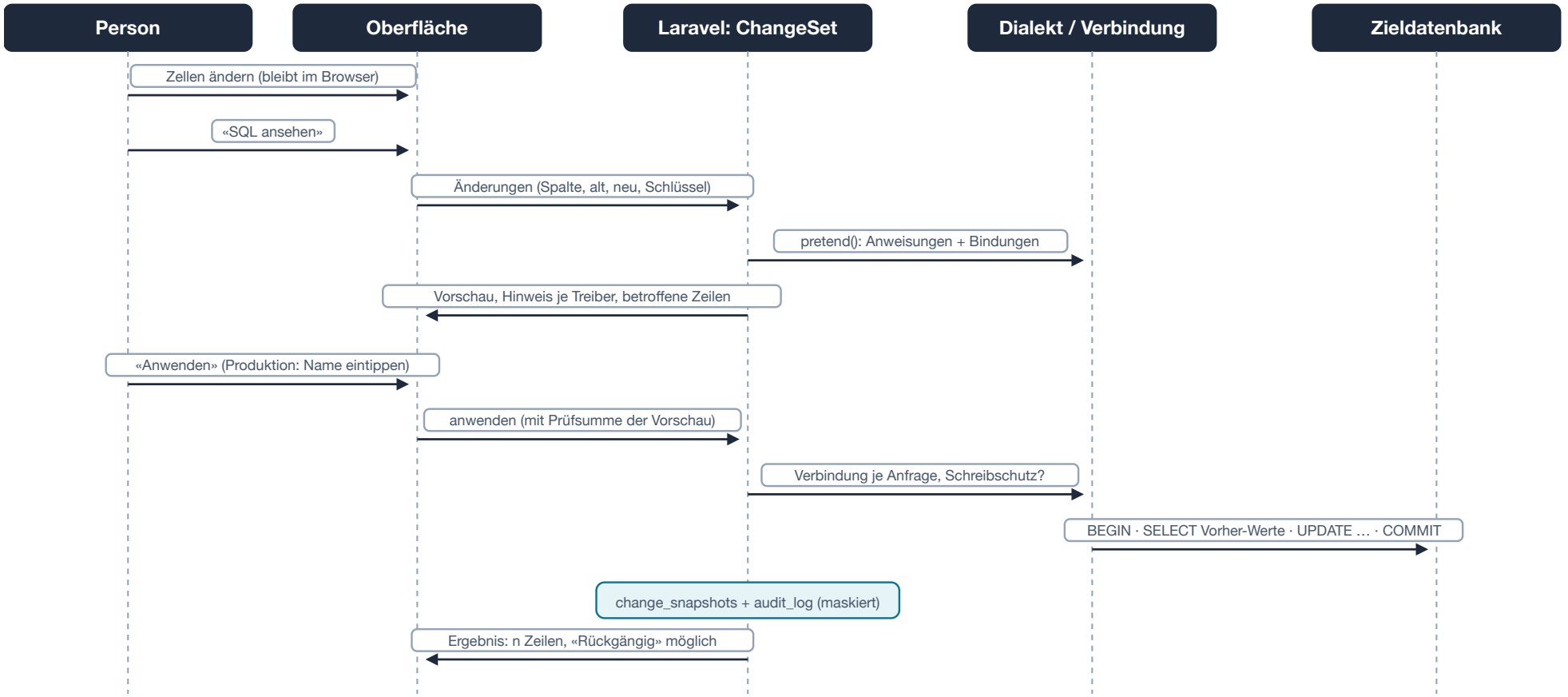
Schicht	Ort (Vorschlag)	Inhalt
Oberfläche	`resources/js/pages/{connections,workspace,account,admin,auth}`, `resources/js/components/{grid,editor,navigator,…}`	Inertia-Seiten und Bausteine aus der Gestaltung § 6; SQL-Editor und Raster als eigene Chunks
HTTP	`app/Http/Controllers/{Connections,Workspace,Account,Admin,Auth}`, `app/Http/Middleware/ResolveTargetConnection`	dünne Controller, Form Requests, Policies
Dienste	`app/Services/{Catalog,DataBrowser,ChangeSet,SchemaEditor,QueryRunner,Transfer,Audit}`	die eigentliche Arbeit, treiberunabhängig
Dialekte	`app/Dialects/{Dialect,MySqlDialect,MariaDbDialect,PostgresDialect,SqliteDialect}`	Vertrag und vier Umsetzungen
Zielverbindungen	`app/Connections/{ConnectionFactory,HostPolicy,ReadOnlyGuard,AccountInspector}`	Verbindung bauen, Hosts prüfen, Schreibschutz, Kontorechte (V-10)
Modelle	`app/Models`	Tabellen aus dem Datenmodell
Hintergrund	`app/Jobs/{RunExport,RunImport,RunSqliteQuery}`	Warteschlange `database`

Neue Ordner liegen **unter** `app/`, kein neuer Basisordner im Repo (Projektregel).

4. Anfrageverlauf

phpdbadmin — Ändern mit Vorschau und Anwenden

Ablauf einer Datenänderung: sammeln im Browser, Vorschau vom Server, Anwenden in einer Transaktion mit Sicherung für Rückgängig



Strukturänderungen laufen gleich, aber die Vorschau kommt aus Blueprint::toSql() und «Anwenden» folgt dem Verfahren je Treiber (ADR-003).

Weicht die Prüfsumme der Vorschau vom neu gebauten SQL ab (jemand hat inzwischen geändert), wird nichts ausgeführt und die Vorschau neu gezeigt.

1. **Anmeldung** (Sitzung) → `ResolveTargetConnection` lädt die Verbindung aus der Adresse und prüft das Recht des Kontos **vor** der Routenbindung — eine fremde Verbindung ist 404, nicht 403 (T-16).
2. Der Controller ruft einen **Dienst**; der Dienst holt über die `ConnectionFactory` eine Verbindung (`DB::build()` mit Zeitzone UTC, Zeitlimit, Schreibschutz) und den passenden **Dialekt**.
3. Der Dienst arbeitet über Laravel (Query Builder, Schema-Introspektion, `Blueprint`) und über den Dialekt.
4. **Querschnitt** hinter den Controllern, nicht in jedem einzeln: Grenzen (Zeit, Zeilen), Maskierung von Meldungen, Protokoll schreibender Aktionen.
5. Die Antwort trägt nie Zugangsdaten (Test über die ausgelieferte Antwort, T-03); die Verbindung wird nach der Anfrage getrennt (`DB::purge`).

5. Zielverbindungen

- **Bauen:** Konfiguration aus `connections` (Passwort entschlüsselt erst hier), `DB::build()` je Anfrage; Name aus Verbindung und Konto, damit sich zwei Konten nie eine Verbindung teilen.
- **Sitzungswerte aus der Konfiguration:** `timezone` (`+00:00` bzw. `UTC`) — belegt in F-2.
- **Zeitlimit je Anweisung:** MySQL `max_execution_time` (gilt nur für SELECT), MariaDB `max_statement_time`, PostgreSQL `statement_timeout`; SQLite hat keines → Abfragen aus dem SQL-Editor laufen in einem **Arbeitsprozess** (`RunSqliteQuery`) mit harter Zeitgrenze (Q-09).
- **Erlaubte Hosts:** `PHPDBADMIN_ALLOWED_HOSTS` (Namen, Adressen, Netze) und `PHPDBADMIN_SQLITE_PATHS`; der Host wird einmal aufgelöst und die geprüfte Adresse verbunden (gegen DNS-Rebinding), SQLite-Pfade mit `realpath()` (gegen `..` und Symlinks) — T-05, T-08.
- **Kontorechte (V-10):** `AccountInspector` liest beim Testen und beim ersten Öffnen Superuser-Status (PostgreSQL `rolsuper`), `FILE`-Recht und `secure_file_priv` (MySQL/MariaDB) und speichert Warnungen in `account_warnings`.
- **Verbindungsfehler** je Treiber in Klartext: MySQL/MariaDB nach Code (2002, 1045, 1049), PostgreSQL nach Meldungstext (alle `08006`) — F-2.

6. Schreibschutz

Belegt in F-7 gegen 5–12 Ausbruchsversuche je Treiber (ADR-004):

Schicht	MySQL / MariaDB	PostgreSQL	SQLite
Konto	Lesekonto empfohlen (Hinweis in der Oberfläche)	dito	—
Je Anweisung	`SET SESSION TRANSACTION READ ONLY` + Transaktion, immer zurückrollen	Transaktion + `SET TRANSACTION READ ONLY`, zurückrollen	Datei mit `Pdo\Sqlite::OPEN_READONLY` geöffnet
Eingabe	eine Anweisung je Aufruf, vorbereitet (`ATTR_EMULATE_PREPARES = false`), nie `unprepared()`	dito	dito, dazu Authorizer (`ATTACH` nur im erlaubten Ordner)

7. Katalog

`Catalog` liest über Laravel (`getSchemas`, `getTables`, `getViews`, `getColumns`, `getIndexes`, `getForeignKeys`) und ergänzt über den Dialekt: Check-Constraints, Trigger, Routinen, eigene Typen (nur PostgreSQL, Array-Typen ausgeblendet), Sequenzen, Events. **Das Schema wird immer ausdrücklich übergeben** — MariaDB liefert sonst alle Datenbanken des Servers (F-1, F-3). Ergebnisse werden je Anfrage berechnet und höchstens 60 Sekunden zwischengespeichert; «Neu laden» leert den Speicher. Typnamen erscheinen im Original und mit einem gemeinsamen Oberbegriff (Text, Zahl, Datum/Zeit, JSON, Binär, Wahrheitswert, Aufzählung).

8. Daten lesen und bearbeiten

- **Blättern über den Schlüssel** (`WHERE (sort, id) > (…) ORDER BY … LIMIT 200`), `OFFSET` nur ohne Primärschlüssel (F-3: 0,1–0,6 ms gegen 18–232 ms). Die nächste Seite wird vorgeladen.
- **Filter** als Struktur (Spalte, Operator, Wert) → Query Builder mit Bindungen; das freie `WHERE` läuft durch den `QueryRunner` mit allen Grenzen.
- **Zählen:** genau bis zu einer Schwelle, sonst Schätzung aus der Statistik (`TABLE_ROWS`, `reltuples`, `sqlite_stat1`), «genau zählen» auf Wunsch.
- `ChangeSet` sammelt Änderungen im Browser; der Server baut daraus Anweisungen, zeigt sie über `pretend()` mit Bindungen als Vorschau (belegt für DML in F-5) und wendet sie in **einer Transaktion** an. Vorher liest er die betroffenen Zeilen und schreibt sie nach `change_snapshots` (Rückgängig, D-09).
- **Tabellen ohne Primärschlüssel:** Bearbeiten nur, wenn die Zeile über alle Spalten eindeutig ist; sonst gesperrt mit Erklärung (D-07).

9. Struktur ändern

Vorschau **immer** aus `Blueprint::toSql()`, nie aus `pretend()` (auf SQLite kaputt, F-5). **Anwenden je Treiber** (ADR-003):

phpdbadmin — «Anwenden» einer Strukturänderung je Treiber (F-5)

Die Vorschau kommt immer aus Blueprint::toSql() — ausgeführt wird genau sie, aber nicht überall gleich

NICHT RÜCKROLLBAR

MySQL und MariaDB

- DDL schliesst jede Transaktion implizit ab
- Anweisung für Anweisung, ohne Transaktion
- Fehler in Schritt 3: Schritte 1–2 bleiben — Vorschau sagt das
- Daten (DML) weiterhin in einer Transaktion

RÜCKROLLBAR

PostgreSQL

- DDL ist transaktional
- alle Anweisungen in einer Transaktion
- Fehler: alles zurück, nichts halb geändert

RÜCKROLLBAR, MIT VERFAHREN

SQLite

- Spalte ändern = Tabelle umbauen (Kopie, Daten, Umbenennen)
- 1 PRAGMA foreign_keys = OFF ausserhalb der Transaktion
- 2 Umbau in der Transaktion
- 3 PRAGMA foreign_key_check
- 4 Commit, foreign_keys = ON

Gemessen, wenn man naiv alles in eine Transaktion legt:

MySQL/MariaDB: die Änderung ist geschrieben, danach wirft Laravel «There is no active transaction» — die Oberfläche hielte sie für gescheitert.
SQLite: PRAGMA foreign_keys wirkt in einer Transaktion nicht; DROP TABLE der Elterntabelle löscht per ON DELETE CASCADE alle Kinder (0 von 2 übrig).
Und: pretend() taugt auf SQLite nicht als Vorschau — die Lesezugriffe liefern leer, heraus kommt «create table … ()».

Belegt in DdlPreviewProbeTest: naiver Weg (Gefahr) und Weg je Treiber (2 von 2 Kindern bleiben) auf allen vier Datenbanken.

Treiber	Verfahren	Was die Vorschau sagt
MySQL, MariaDB	Anweisung für Anweisung ohne Transaktion; bei Fehler stoppen und den Stand melden	«Nicht rückrollbar: bei einem Fehler bleiben die vorherigen Schritte bestehen.»
PostgreSQL	alle Anweisungen in einer Transaktion	«Wird in einer Transaktion ausgeführt.»
SQLite	`PRAGMA foreign_keys = OFF` ausserhalb, Umbau in der Transaktion, `PRAGMA foreign_key_check`, Commit, `ON`	«Tabelle wird umgebaut (Kopie); in einer Transaktion.»

Nach eigenen Importen und auf Knopfdruck aktualisiert der Dienst die Statistik (`ANALYZE`, E-06) — ohne sie sortieren MySQL und MariaDB über einen Index hundertmal langsamer (F-3).

10. SQL-Editor

- **Aufteilen in Anweisungen** auf dem Server mit einem Zerleger je Dialekt, der Zeichenketten, Kommentare, `$$`-Blöcke (PostgreSQL) und `BEGIN … END` in Routinen kennt; jede Anweisung wird einzeln vorbereitet und ausgeführt. Der Zerleger ist das grösste technische Risiko des Editors (Probe zu Beginn von TB 03).
- **Gefährliche Anweisungen** (DROP, TRUNCATE, DELETE/UPDATE ohne WHERE) erkennt derselbe Zerleger; sie brauchen die Bestätigung (Q-07).
- **Zeilengrenze ohne Speicherexplosion** (F-8): MySQL/MariaDB ungepuffert lesen und nach N Zeilen mit `KILL QUERY` beenden; PostgreSQL bei SELECT über einen Server-Cursor (`DECLARE … CURSOR` in der Transaktion, `FETCH N`), weil libpq sonst das ganze Ergebnis hält; SQLite streamt von selbst.
- **Abbrechen:** Der Lauf merkt sich Sitzungskennung (`CONNECTION_ID()`, `pg_backend_pid()`) unter einer Laufkennung im Cache; «Abbrechen» ist eine zweite Anfrage, die `KILL QUERY` bzw. `pg_cancel_backend()` über eine eigene Verbindung schickt und einen Merker setzt. Weil MySQL still abbricht (Ergebnis `1`, kein Fehler), entscheidet der Merker über die Anzeige «abgebrochen», nicht der Fehlercode (Q-02). SQLite: Arbeitsprozess beenden.
- **Editor im Browser:** CodeMirror 6 mit `@codemirror/lang-sql`, Dialekt und Schema aus dem Katalog, als eigener Chunk (425 KB, 138 KB gzip) nur auf dem SQL-Reiter (ADR-005).

11. Export und Import

Export in Paketen über den Schlüssel (`lazyById(5000)`, gemessen 3 MB je Million Zeilen) im Hintergrund (`RunExport`), Datei mit Frist; ohne laufenden Worker synchron bis zu einer Grenze (B-07). Import: CSV mit Spaltenzuordnung, **Probelauf** in einer Transaktion, die zurückgerollt wird; danach echter Lauf in Paketen zu 1'000 (gemessen 143–502 ms je 100'000 Zeilen) und `ANALYZE` (ADR-007).

12. Anmeldung und Konten

Fortify nur noch für die Passkey-Zeremonie; seit TB 00 registriert Fortify **keine eigenen Routen** (`Fortify::ignoreRoutes()`), `routes/auth.php` trägt nur die Passkey-Wege der Bibliothek — einen Passwort-Endpunkt gibt es damit gar nicht (strenger als das ursprünglich geplante `authenticateUsing()`, das nur abgelehnt hätte). Die Passwortbestätigung (`password.confirm`) wird durch Bestätigung per Passkey oder per Code an die eigene Adresse ersetzt; jede Anmeldung gilt zugleich als Bestätigung (F-6, ADR-008). Code-Anmeldung als eigener Controller nach dem Muster von

Basalto: Code mit HMAC gespeichert, benannte Limiter je Adresse und IP, gleiche Antwort für unbekannte Adressen, Sitzung neu. Erstes Konto per `php artisan phpdbadmin:install`, weitere per Einladung. Seit TB 00 erkennt `/login` eine Installation ohne Konto und bietet die Ersteinrichtung an: im Browser mit dem Einrichtungsschlüssel aus einer Datei auf dem Server (T-19) oder per Befehl, der einen signierten Link ausgibt (30 Minuten, gilt nur bis zur ersten Anmeldung). Wer sich ohne Passkey anmeldet, landet im Onboarding, das gleich einen Passkey anlegt. Rollen `admin` / `member`, Rechte je Verbindung über Policies.

13. Protokoll und Maskierung

`Audit` schreibt jede schreibende Aktion mit Anweisung, Ziel und betroffenen Zeilen. **Fehlermeldungen werden maskiert**, bevor sie angezeigt, protokolliert oder gemeldet werden: Laravel setzt gebundene Werte in die Meldung ein (F-7) — der `Audit`-Dienst und ein Exception-Reporter ersetzen sie durch Platzhalter. Das Laravel-Log bekommt keine Werte aus Zieldatenbanken.

14. Qualitätssicherung

- **Tests auf vier Treibern:** App-Datenbank in Tests SQLite im Speicher; Tests gegen Zieldatenbanken laufen gegen die Herd-Dienste (MySQL 8.4 :3307, MariaDB 10.11 :3306, PostgreSQL 18 :5432, SQLite) und überspringen einen nicht erreichbaren Server ausdrücklich — aus den Proben der Feasibility werden **Vertragstests** des Dialekts.
- **Pipeline sparsam** (globale Regel): je Push ein Auftrag (Pint, Larastan Level 7, `vp check`, `vue-tsc`, Tests auf SQLite); die Matrix mit MySQL, MariaDB und PostgreSQL als Service-Container wöchentlich und von Hand; `paths-ignore` für Doku.
- **Durchlauf im Browser** je Timebox gegen die Wireframes, bei 1440, 768 und 375 px; Sicherheitsprüfung nach OWASP vor v1.0.

15. Entscheide

Nr.	Entscheid	ADR
A-01	Dialekt-Schicht: ein Vertrag, vier Umsetzungen; Laravel zuerst, Schema immer ausdrücklich	ADR-001
A-02	Zielverbindungen je Anfrage über `DB::build()`, Sitzungswerte aus der Konfiguration	ADR-002
A-03	Vorschau aus `toSql()`; Anwenden je Treiber	ADR-003
A-04	Schreibschutz in drei Schichten	ADR-004
A-05	SQL-Editor CodeMirror 6, eigener Chunk	ADR-005
A-06	Raster mit `@tanstack/vue-virtual`, Blättern über den Schlüssel	ADR-006
A-07	Export und Import in Paketen im Hintergrund, Statistik nach Import	ADR-007
A-08	Anmeldung ohne Passwort: Fortify nur für Passkeys, eigener Code-Login	ADR-008
A-09	App-Datenbank SQLite als Vorgabe	ADR-009
A-10	Lizenz MIT, Abhängigkeiten MIT-verträglich	ADR-010
A-11	PHP 8.5 als Mindestversion	ADR-011
A-12	Anweisungen auf dem Server zerlegen, eine je Aufruf	ADR-004
A-13	Zeilengrenze je Treiber: ungepuffert + `KILL QUERY`, Server-Cursor, Streaming	ADR-002

Nr.	Entscheid	ADR
A-14	Abbruch über einen eigenen Merker, nicht über den Fehlercode	ADR-002

16. Offene Punkte

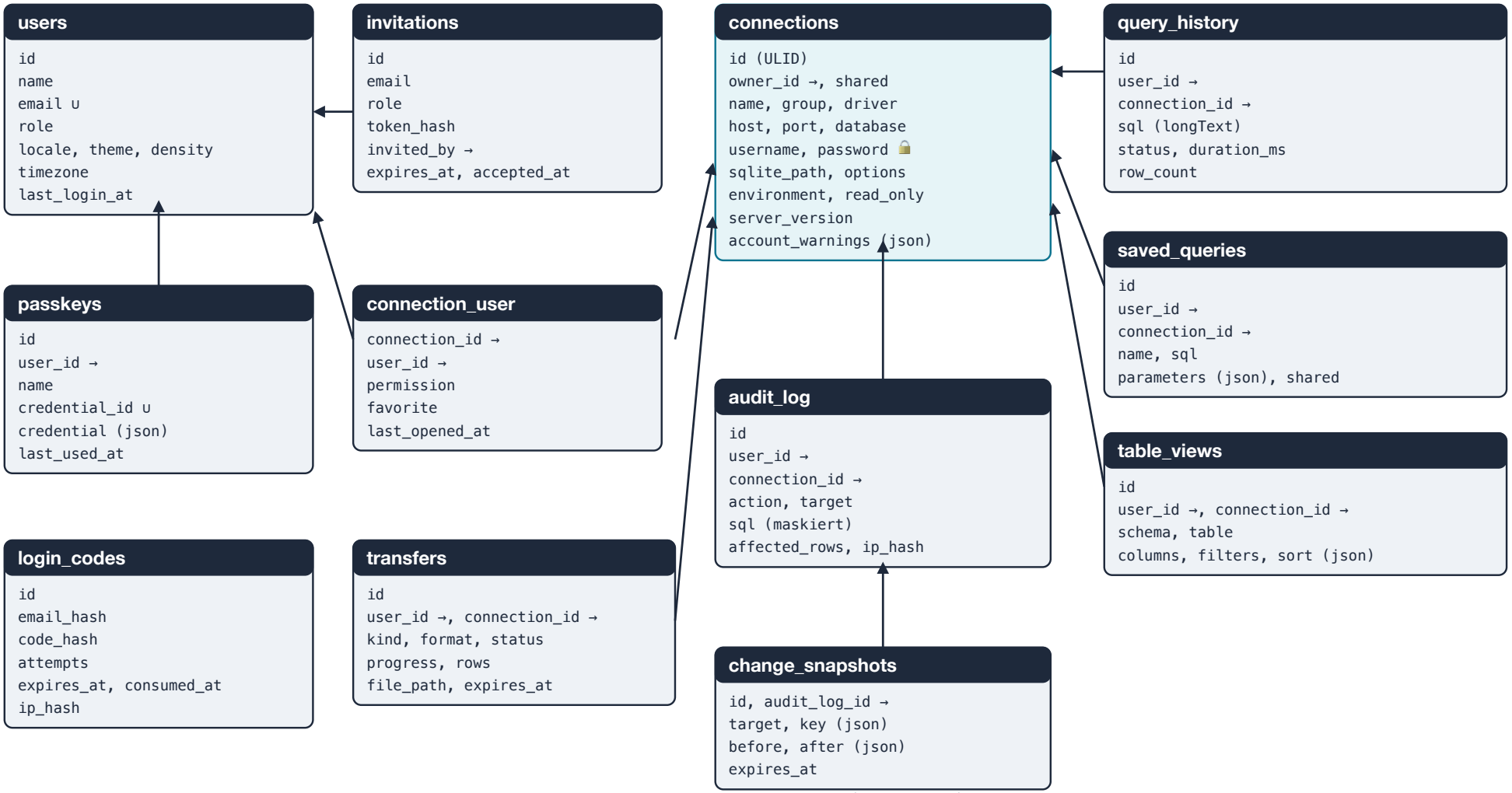
- Zerleger für SQL-Anweisungen: eigener Bau oder Paket — Probe zu Beginn von TB 03, Lizenz MIT-verträglich.
- SSH-Tunnel (V-08, Stufe 4): PHP-Erweiterung `ssh2` oder ein Hilfsprozess `ssh -L` — Entscheid in TB 11.
- Gemeinsamer Oberbegriff der Typen: Liste je Dialekt in TB 01 festlegen.

Anhang D — Datenmodell — App-Datenbank v0.1

Stand: 28.10.2026 · v0.1 (Foundation). Beschreibt die **eigene Datenbank von phpdbadmin** — Konten, Passkeys, Verbindungen, Rechte, Verlauf, Protokoll. Die verwalteten Datenbanken sind Umsysteme und haben hier kein Modell. Vorgabe ist SQLite (ADR-009); MariaDB, MySQL und PostgreSQL sind möglich (B-06), deshalb gelten die Regeln in § 1 für alle vier.

phpdbadmin — App-Datenbank v0.1 (ER)

Eigene Tabellen von phpdbadmin; Zieldatenbanken haben hier kein Modell. Zeitpunkte UTC, Geheimnisse verschlüsselt, Einmalwerte als Prüfsumme



Pfeile: Fremdschlüssel von der verweisenden zur referenzierten Tabelle; die Arbeitstabellen verweisen zusätzlich über user_id auf users (nicht gezeichnet).

∪ eindeutig · 🔒 verschlüsselt (encrypted-Cast) · Fristen: Codes 1 Tag, Verlauf 90 Tage, Sicherungen 7 Tage, Exporte 24 h, Protokoll 365 Tage

1. Grundsätze

1. **Kein Passwort** für Konten: `users` hat keine Spalten `password`, `remember_token`, `two_factor_*`; die Tabelle `password_reset_tokens` entfällt (A-01).
2. **Geheimnisse verschlüsselt** mit dem `encrypted`-Cast (`APP_KEY`): Passwörter der Zielverbindungen, Schlüssel für SSH und KI. Ein Wechsel des `APP_KEY` macht sie unlesbar — der Betrieb kennt `APP_PREVIOUS_KEYS` (B-05).
3. **Nur Prüfsummen für Einmalwerte:** Anmeldecodes und Einladungstoken als Hash (`hash_hmac` mit `APP_KEY`), nie im Klartext; IP-Adressen als Prüfsumme.
4. **Zeitpunkte** als `dateTime` in UTC, nie `timestamp NOT NULL` ohne Vorgabe (MariaDB 10.6 setzt sonst still `ON UPDATE CURRENT_TIMESTAMP`); Anzeige in der Zeitzone des Kontos.
5. **Lange Texte** (SQL, Fehlermeldungen, Vorher-Werte) als `longText` — `TEXT` endet auf MySQL/MariaDB bei 64 KB.
6. **JSON** als `json`; Tests vergleichen JSON mit `assertEquals`, nie mit `assertSame` (MySQL ordnet Schlüssel um).
7. **Kennungen:** `connections` mit ULID (die Kennung steht in Adressen und darf nichts über die Anzahl verraten); übrige Tabellen mit fortlaufender Nummer.
8. **Aufbewahrung** beim Anlegen mitgebaut: Verlauf, Protokoll, Rückgängig-Sicherungen und Exportdateien haben eine Frist und einen täglichen Aufräumlauf (`model:prune`).
9. **Kein Datenbankfehler ins eigene Protokoll** schreiben, wenn das Protokoll selbst in der App-Datenbank liegt (Rückkopplung) — Fehler der App-Datenbank gehen nur ins Log.

2. Tabellen

2.1 Konten und Anmeldung

Tabelle	Spalten (Auswahl)	Zweck
`users`	`id`, `name`, `email` (eindeutig, klein geschrieben), `role` (`admin` · `member`), `locale`, `theme` (`system` · `light` · `dark`), `density` (`default` · `compact`), `timezone`, `last_login_at`, Zeitstempel	Konto ohne Passwort; Name und E-Mail ändert ein Administrator (A-05)
`passkeys`	`id`, `user_id`, `name`, `credential_id` (eindeutig), `credential` (json), `last_used_at`, Zeitstempel	aus `laravel/passkeys`, unverändert (A-03)
`login_codes`	`id`, `email_hash`, `code_hash`, `attempts`, `expires_at`, `consumed_at`, `ip_hash`, `created_at`	Anmeldung per Code: sechs Ziffern, 10 Minuten, einmalig (A-02); unbekannte Adressen bekommen keinen Datensatz, aber dieselbe Antwort
`invitations`	`id`, `email`, `role`, `token_hash`, `invited_by`, `expires_at`, `accepted_at`, Zeitstempel	Einladung statt Registrierung (A-06)
`sessions`	wie Laravel	Sitzungen sehen und beenden (A-08)

2.2 Verbindungen und Rechte

Tabelle	Spalten (Auswahl)	Zweck
`connections`	`id` (ULID), `owner_id`, `shared` (bool), `name`, `group`, `driver` (`mysql` · `mariadb` · `pgsql` · `sqlite`), `host`, `port`, `database`, `username`, `password` (verschlüsselt, leer bei «nicht speichern»), `store_password` (bool), `sqlite_path`, `options` (json: SSL-Modus, Zeichensatz, Suchpfad), `environment` (`local` · `test` · `production`), `read_only` (bool), `server_version`, `account_warnings` (json, V-10), `last_tested_at`, Zeitstempel	Zielverbindung (V-01 bis V-05, V-09, V-10)
`connection_user`	`connection_id`, `user_id`, `permission` (`read` · `write` · `manage`), `favorite` (bool), `last_opened_at`	Recht je Verbindung und Konto, Favoriten (A-07, V-06)

Regeln: Produktion ist beim Anlegen `read_only = true` (V-04). Eine persönliche Verbindung (`shared = false`) sieht nur ihr Besitzer; geteilte Verbindungen nur, wer in `connection_user` steht (V-09). `host` und `sqlite_path` werden beim Speichern **und** beim Verbinden gegen die erlaubten Hosts und Ordner geprüft (V-05).

2.3 Arbeit: Verlauf, gespeicherte Abfragen, Ansichten

Tabelle	Spalten (Auswahl)	Zweck
`query_history`	`id`, `user_id`, `connection_id`, `sql` (longText), `status` (`ok` · `error` · `cancelled` · `timeout`), `duration_ms`, `row_count`, `created_at`	Verlauf je Konto und Verbindung (Q-04); Frist 90 Tage
`saved_queries`	`id`, `user_id`, `connection_id` (nullable), `name`, `sql`, `parameters` (json), `shared` (bool), Zeitstempel	gespeicherte Abfragen (Q-04)
`table_views`	`id`, `user_id`, `connection_id`, `schema`, `table`, `columns` (json: Reihenfolge, Breite, ausgeblendet), `filters` (json), `sort` (json), Zeitstempel	gemerkte Ansicht je Tabelle (D-01, D-02) — gespeichert wird die Wahl, nicht das Ergebnis

2.4 Änderungen, Protokoll, Austausch

Tabelle	Spalten (Auswahl)	Zweck
`audit_log`	`id`, `user_id`, `connection_id`, `action` (z. B. `data.update`, `schema.alter`, `sql.execute`, `auth.login`), `target` (Schema.Tabelle), `sql` (longText, Werte maskiert), `affected_rows`, `ip_hash`, `created_at`	Protokoll aller schreibenden Aktionen und Anmeldungen (B-04, A-09); Frist einstellbar, Vorgabe 365 Tage
`change_snapshots`	`id`, `audit_log_id`, `user_id`, `connection_id`, `target`, `key` (json: Primärschlüssel), `before` (json), `after` (json), `expires_at`	Rückgängig für Datenänderungen (D-09): Vorher-Werte bis zur Grenze (Vorgabe 1'000 Zeilen je Änderung, 7 Tage)
`transfers`	`id`, `user_id`, `connection_id`, `kind` (`export` · `import`), `format`, `status` (`queued` · `running` · `done` · `failed`), `progress`, `rows`, `file_path`, `error`, `expires_at`, Zeitstempel	Export und Import im Hintergrund (E-01, E-02); Dateien unter `storage/app/private/transfers`, Frist 24 Stunden
`jobs`, `failed_jobs`, `cache`	wie Laravel	Warteschlange, Abbruch-Merker laufender Abfragen (Q-02)

3. Was bewusst fehlt

- **Kein Abbild der Zieldatenbanken:** Katalog, Zeilenzahlen und Struktur werden je Anfrage gelesen, höchstens kurz zwischengespeichert (Cache mit Ablauf) — nie in Tabellen, die veralten.
- **Kein Klartext gebundener Werte im Protokoll:** `audit_log.sql` speichert die Anweisung mit Platzhaltern bzw. maskierten Werten; die Werte selbst stehen nur in `change_snapshots` (mit Frist) — so bleibt Rückgängig möglich, ohne dass das Protokoll Daten sammelt.
- **Keine Rechte auf Tabellenebene:** dafür sind die Konten der Datenbank-Server da (Sicherheit und Bedrohungen § 3).

4. Löschen und Aufräumen

Was	Wann	Wie
Anmeldecodes	nach Verbrauch oder 10 Minuten, Datensatz nach 1 Tag	`model:prune`
Einladungen	nach Annahme oder Ablauf (7 Tage), Datensatz nach 30 Tagen	`model:prune`
Verlauf	90 Tage	`model:prune`
Rückgängig-Sicherungen	7 Tage	`model:prune`
Exportdateien	24 Stunden	Aufräumlauf löscht Datei und setzt `file_path` leer
Protokoll	Einstellung, Vorgabe 365 Tage	`model:prune`
Konto gelöscht	Passkeys, Sitzungen, persönliche Verbindungen, Verlauf, gespeicherte Abfragen mit; Protokolleinträge bleiben mit `user_id = null` und Name im Eintrag	Fremdschlüssel `cascadeOnDelete` bzw. `nullOnDelete`

5. Offene Punkte

- Einstellbare Fristen: global in der Konfiguration oder im Verwaltungsbereich? Vorschlag: Konfiguration (`.env`), angezeigt in der Verwaltung (TB 07).
- KI-Schlüssel (Q-08, Stufe 4) — Tabelle oder Spalte am Konto, erst in TB 12.

Anhang E — PRL — Prioritised Requirements List (phpdbadmin)

> **Status:** v1.0 (28.10.2026) — erzeugt aus CR-001 § 4 (81 Anforderungen) mit `werkzeuge/prl-bauen.py` ; **nicht von Hand bearbeiten**, Zuordnung und Grössen stehen im Skript. Jede Anforderung hat eine Timebox (Timebox-Plan), eine Grösse (S 1 · M 2 · L 3 · XL 5 Punkte) und eine **Priorität im Plan**: Must-Anforderungen des CR mit einem Rückfallweg laufen als *Should* ↓ und rutschen bei Zeitnot in die nächste Stufe. Spalte «Feasibility» nennt, was schon belegt ist.

1. Kennzahlen je Stufe

Stufe	Anforderungen	Punkte	davon Must im Plan	Must-Anteil	Must im CR
0 Fundament	16	31	28	**90 %** ⚠️	15 von 16
1 Lesen (Alpha)	23	49	28	**57 %** ✅	15 von 23
2 Bearbeiten (v1.0)	24	62	27	**44 %** ✅	12 von 24
3 Verwalten (v1.1)	12	27	0	**0 %** ✅	0 von 12
4 Komfort und Team (v1.2)	6	14	0	**0 %** ✅	0 von 6
bis v1.0 (Stufen 0–2)	63	142	83	**58 %** ✅	42 von 63

Stufe 0 bleibt über 60 Prozent — bewusst: Das Fundament hat keinen Rückfallweg. Ohne Anmeldung, Verbindungen, Dialekt-Schicht und Tests auf vier Datenbanken gibt es keine Stufe 1; die Entfernung von Passwort, Dashboard und Profil ist eine ausdrückliche Vorgabe. Der Puffer liegt deshalb in den Stufen 1 und 2; bis v1.0 zusammen bleibt der Must-Anteil unter 60 Prozent.

Personentage je Timebox (Budget je Stufe nach CR-001 § 9, obere Schätzung 53 PT, anteilig nach Punkten):

TB	Titel	Stufe	Anforderungen	Punkte	Must	PT
00	Fundament: Starter-Kit, Anmeldung, Verbindungen, Dialekt-Schicht	0	16	31	28	~8,0
01	Katalog und Navigator	1	9	18	11	~4,0
02	Datenraster und Filter	1	8	16	8	~3,6
03	SQL-Editor und Befehlspalette → Alpha	1	6	15	9	~3,4
04	Bearbeiten mit Vorschau und Rückgängig	2	8	21	10	~4,7
05	Struktur ändern	2	4	12	8	~2,7
06	Export und Import	2	4	9	3	~2,0
07	Konten, Rechte, Teilen, Protokoll	2	3	8	3	~1,8
08	Sicherheit, Barrierefreiheit, Installation → v1.0	2	5	12	3	~2,7
09	Server-Verwaltung	3	5	9	0	~3,3
10	Weitere Objekte, Analyse, Sicherung → v1.1	3	7	18	0	~6,7
11	Komfort: Schemavergleich, Laravel, Herd, SSH	4	4	10	0	~7,1

TB	Titel	Stufe	Anforderungen	Punkte	Must	PT
12	Team und KI → v1.2	4	2	4	0	~2,9

2. Anforderungen je Timebox

TB 00 — Fundament: Starter-Kit, Anmeldung, Verbindungen, Dialekt-Schicht (Stufe 0; 16 Anforderungen, 31 Punkte, Must 28)

Nr.	Anforderung	CR	Plan	Grösse	Feasibility / Rückfall	Abnahme (Kurzform, vollständig im CR)
A-01	Passwort entfernt	Must	**Must**	M	F-6: Fortify-Routen gelesen, Muster Basalto	Routenliste ohne diese Routen bzw. Fortify-Endpunkte lehnen ab
A-02	Anmeldung mit Code per Mail	Must	**Must**	L	F-6: Vorlage Basalto (LoginController)	Tests für Erfolg, Ablauf, Wiederverwendung, falschen Code, Drosselung (benannter Limiter), unbekann…
A-03	Anmeldung mit Passkey	Must	**Must**	M	F-6: laravel/passkeys 0.2.1, Software-Authenticator	Zeremonie mit virtuellem Authenticator im Test
A-04	Dashboard entfernt	Must	**Must**	S	—	`/dashboard` 404
A-05	Profilseite entfernt	Must	**Must**	S	—	`/settings/profile` 404
A-06	Keine offene Registrierung	Must	Should ↓	M	Rückfall: Konten per Konsole bis zur Einladung	Test: Einladung annehmen, abgelaufene ablehnen
A-09	Anmeldungen und Fehlversuche im Protokoll	Should	Should	S	—	Eintrag mit Zeit, Konto bzw. Adresse (gehasht), IP-Prüfsumme
V-01	Verbindung anlegen für MariaDB, MySQL, PostgreSQL	Must	**Must**	L	F-1/F-2: DB::build() auf vier Treibern	je Treiber ein Test gegen den echten Dienst
V-02	Verbindung testen	Must	**Must**	M	F-2: Fehlerfälle je Treiber gemessen	je Fehlerart ein Test mit erzwungenem Fehler
V-03	Zugangsdaten verschlüsselt	Must	**Must**	M	—	Test: Inertia-Props und JSON enthalten kein Passwort
V-05	Erlaubte Hosts und SQLite-Ordner	Must	**Must**	M	F-7: Authorizer für SQLite-ATTACH	Test mit nicht erlaubtem Host und Pfad (auch `..`, Symlink)
B-01	Stack	Must	**Must**	S	Pakete probeweise installiert (F-4)	Routenliste und Seitenordner ohne Starter-Kit-Reste
B-02	Dialekt-Schicht	Must	**Must**	XL	F-1/F-5: Proben werden Vertragstests	Vertragstest läuft gegen alle vier Umsetzungen
B-03	Tests gegen vier Datenbanken	Must	**Must**	M	MySQL 8.4.11 in Herd; Proben laufen auf vier Treibern	lokales Skript fährt alle vier
B-06	App-Datenbank SQLite als Vorgabe, MariaDB, MySQL und PostgreSQL möglich	Must	**Must**	S	—	Suite läuft mit jeder App-Datenbank

Nr.	Anforderung	CR	Plan	Grösse	Feasibility / Rückfall	Abnahme (Kurzform, vollständig im CR)
B-08	Qualität	Must	**Must**	S	—	lokale Prüfkette in der Reihenfolge der CI

TB 01 — Katalog und Navigator (Stufe 1; 9 Anforderungen, 18 Punkte, Must 11)

Nr.	Anforderung	CR	Plan	Grösse	Feasibility / Rückfall	Abnahme (Kurzform, vollständig im CR)
V-04	Umgebung	Must	**Must**	M	F-7: Wächter gegen Ausbruchsversuche	Test: schreibgeschützte Verbindung lehnt jede schreibende Anweisung ab, auch die Ausbruchsversuche…
V-06	Verbindungen gruppieren, sortieren, suchen, Favoriten	Should	Should	S	—	Durchlauf
V-10	Warnung bei Konten mit zu vielen Rechten	Should	Should	M	F-7: Kontorechte gelesen (MariaDB, PostgreSQL)	Test gegen Herd-`root` auf MariaDB und PostgreSQL (warnt) und ein Lesekonto (warnt nicht) — F-7
O-01	Navigator	Must	**Must**	L	F-3: Katalog mit 1'000 Tabellen 1–13 ms	Durchlauf mit 1'000 Tabellen flüssig (N-01)
O-04	Gleiche Bildschirme und Begriffe	Must	**Must**	M	—	Bildschirmfotos derselben Tabelle in allen vier Datenbanken nebeneinander
O-06	Hell und dunkel	Must	Should ↓	M	Rückfall: hell und Deutsch zuerst	Sprachdateien vollständig (Test über alle Schlüssel, ohne Rückfall)
S-01	Struktur ansehen	Must	**Must**	L	F-1: Introspektion und Dialekt-Abfragen	Tests je Treiber gegen eine Beispieltabelle mit allen Merkmalen
N-04	Kompatibilität	Must	Should ↓	M	Rückfall: aktuelle Versionen lokal, Untergrenzen in der Wochen-Matrix	Feasibility prüft die Untergrenzen
N-06	Sprache	Must	**Must**	S	—	Test über Sprachdateien

TB 02 — Datenraster und Filter (Stufe 1; 8 Anforderungen, 16 Punkte, Must 8)

Nr.	Anforderung	CR	Plan	Grösse	Feasibility / Rückfall	Abnahme (Kurzform, vollständig im CR)
O-03	Reiter	Should	Should	M	—	Durchlauf
O-05	Tastatur	Should	Should	S	—	Durchlauf nur mit Tastatur
O-09	Leer-, Fehler- und Ladezustände mit Ursache und nächstem Schritt	Must	Should ↓	M	Rückfall: Fehlerkarte zuerst, Leer- und Ladezustände danach	Durchlauf mit leerer Tabelle, SQL-Fehler, getrennter Verbindung, abgelaufener Sitzung

Nr.	Anforderung	CR	Plan	Grösse	Feasibility / Rückfall	Abnahme (Kurzform, vollständig im CR)
D-01	Datenraster	Must	**Must**	L	F-3: Blättern über den Schlüssel 0,1–0,6 ms; Raster-Probe	Tests je Treiber mit Sortierung inkl. NULL (Reihenfolge ausdrücklich, nicht treiberabhängig)
D-02	Filtern	Must	**Must**	L	—	Tests je Treiber, auch mit Umlauten und Werten, die wie SQL aussehen
D-03	Fremdschlüssel folgen	Should	Should	M	—	Test je Treiber
D-06	Zeilen zählen	Should	Should	S	F-3: Schätzung innerhalb 0,4 %	Test mit Schwelle
N-01	Leistung	Must	**Must**	M	F-3: erste Seite 0,1–0,6 ms (DB), 281 ms (Browser)	Messung mit erzeugten Daten je Treiber, Zeit im Protokoll der Timebox

TB 03 — SQL-Editor und Befehlspalette → Alpha (Stufe 1; 6 Anforderungen, 15 Punkte, Must 9)

Nr.	Anforderung	CR	Plan	Grösse	Feasibility / Rückfall	Abnahme (Kurzform, vollständig im CR)
O-02	Befehlspalette ⌘K	Must	Should ↓	M	Rückfall: Navigator	Test der Suchquelle
Q-01	SQL-Editor	Must	**Must**	L	F-4: CodeMirror 6 installiert, Vervollständigung belegt	Durchlauf je Treiber
Q-02	Ausführen	Must	**Must**	L	F-2: Abbrechen 22–29 ms, MySQL still	Test: Abbrechen beendet die Abfrage auf dem Server
Q-03	Grenzen	Must	**Must**	L	F-2/F-7: Zeitlimits und Schreibschutz belegt	Tests je Treiber mit langer und mit schreibender Anweisung auf geschützter Verbindung — F-2, F-7
Q-04	Verlauf je Konto und Verbindung	Should	Should	M	—	Test
Q-09	SQLite-Abfragen aus dem SQL-Editor in einem Arbeitsprozess mit Zeitgrenze	Should	Should	M	F-2: SQLite ohne Zeitlimit gemessen	Test: eine Abfrage über die Grenze wird beendet, die Oberfläche meldet es — F-2

TB 04 — Bearbeiten mit Vorschau und Rückgängig (Stufe 2; 8 Anforderungen, 21 Punkte, Must 10)

Nr.	Anforderung	CR	Plan	Grösse	Feasibility / Rückfall	Abnahme (Kurzform, vollständig im CR)
V-11	Schreibschutz auf Zeit aufheben	Should	Should	M	Wireframes 5–7; Rückfall: Schreibschutz in der Verbindung ausschalten	Test: Freigabe läuft nach 15 Minuten ab (Uhr gestellt), Grund und Ende im Protokoll, ohne Recht 403…

Nr.	Anforderung	CR	Plan	Grösse	Feasibility / Rückfall	Abnahme (Kurzform, vollständig im CR)
D-04	Bearbeiten	Must	**Must**	XL	F-5: pretend() für DML belegt	Test je Treiber: nichts geschrieben vor «Anwenden»
D-05	Werte typgerecht	Must	Should ↓	L	Rückfall: Werte als Text bearbeiten, Bildvorschau später	Tests je Typ und Treiber, Werte nach dem Speichern byte-genau verglichen
D-07	Tabellen ohne Primärschlüssel	Must	Should ↓	M	Rückfall: Tabellen ohne Primärschlüssel nur lesen	Test: mehrdeutige Zeile wird nicht geändert
D-09	Rückgängig für Datenänderungen	Should	Should	L	—	Test: Änderung anwenden, rückgängig, Zustand gleich dem Ausgangszustand
Q-07	Gefährliche Anweisungen	Must	**Must**	M	—	Tests mit Varianten (Kommentare, Gross-/Kleinschreibung, mehrere Anweisungen)
E-05	Kopieren aus dem Raster als TSV, Markdown oder INSERT	Should	Should	S	—	Test
N-03	Kein Datenverlust durch die Anwendung	Must	**Must**	L	F-5: naiver Weg verliert Kinder, Weg je Treiber nicht	Tests mit erzwungenem Fehler mitten in einer Folge und mit Elterntabelle und Kindern (naiv: 0 von 2…

TB 05 — Struktur ändern (Stufe 2; 4 Anforderungen, 12 Punkte, Must 8)

Nr.	Anforderung	CR	Plan	Grösse	Feasibility / Rückfall	Abnahme (Kurzform, vollständig im CR)
S-02	Tabelle anlegen	Must	**Must**	L	F-5: toSql() = Ausführung	Test: Vorschau (aus `Blueprint::toSql()`, nie `pretend()`) = ausgeführtes SQL
S-03	Tabelle ändern	Must	**Must**	XL	F-5: Anwenden je Treiber belegt	Tests je Treiber mit Elterntabelle und abhängigen Zeilen: Daten, Indizes, Fremdschlüssel und Kinder…
S-04	Tabelle umbenennen, kopieren	Must	Should ↓	M	Rückfall: im SQL-Editor mit Bestätigung (Q-07)	Test: ohne korrekten Namen keine Ausführung
S-05	Datenbank bzw. Schema anlegen und löschen	Should	Should	M	—	Test je Treiber (SQLite: neue Datei im erlaubten Ordner)

TB 06 — Export und Import (Stufe 2; 4 Anforderungen, 9 Punkte, Must 3)

Nr.	Anforderung	CR	Plan	Grösse	Feasibility / Rückfall	Abnahme (Kurzform, vollständig im CR)
E-01	Export	Must	**Must**	L	F-8: lazyById 3 MB je Million Zeilen	Test: Export und Re-Import ergeben dieselben Daten je Treiber

Nr.	Anforderung	CR	Plan	Grösse	Feasibility / Rückfall	Abnahme (Kurzform, vollständig im CR)
E-02	Import	Must	Should ↓	L	F-8: 100'000 Zeilen 143–502 ms; Rückfall: SQL-Datei im Editor	Tests mit Fehlerzeilen
E-06	Statistik aktualisieren	Should	Should	S	F-3: Gegenprobe EXPLAIN vor und nach ANALYZE	Test: nach Import ist die Schätzung der Zeilenzahl aktuell — F-3
B-07	Queue für Export und Import	Should	Should	M	—	Test beider Wege

TB 07 — Konten, Rechte, Teilen, Protokoll (Stufe 2; 3 Anforderungen, 8 Punkte, Must 3)

Nr.	Anforderung	CR	Plan	Grösse	Feasibility / Rückfall	Abnahme (Kurzform, vollständig im CR)
A-07	Rollen und Rechte	Should	Should	L	—	Policy-Tests je Recht
V-09	Verbindungen persönlich oder geteilt	Should	Should	M	—	Test: persönliche Verbindung ist für andere 404
B-04	Protokoll	Must	**Must**	L	F-7: Meldungen tragen Werte — Maskierung nötig	Test: jede schreibende Aktion erzeugt genau einen Eintrag

TB 08 — Sicherheit, Barrierefreiheit, Installation → v1.0 (Stufe 2; 5 Anforderungen, 12 Punkte, Must 3)

Nr.	Anforderung	CR	Plan	Grösse	Feasibility / Rückfall	Abnahme (Kurzform, vollständig im CR)
O-07	Auf Tablet und Telefon lesbar, Bearbeiten am Desktop	Should	Should	L	—	Durchlauf bei 375, 768 und 1440 px
O-08	WCAG 2.2 AA in den Kernbildschirmen	Should	Should	M	—	axe ohne Befund
B-05	Installation per Git oder composer create-project und einem Einrichtungsbefehl	Should	Should	M	—	Durchlauf in einem leeren Ordner unter 15 Minuten
N-02	Sicherheit	Must	**Must**	L	Bedrohungsmodell T-01 bis T-18	Prüfbericht ohne hohen Befund
N-05	Kurze Hilfe in der Oberfläche	Should	Should	M	—	Doku-Prüfung im Commit

TB 09 — Server-Verwaltung (Stufe 3; 5 Anforderungen, 9 Punkte, Must 0)

Nr.	Anforderung	CR	Plan	Grösse	Feasibility / Rückfall	Abnahme (Kurzform, vollständig im CR)
W-01	Server-Übersicht	Should	Should	M	—	Test je Treiber
W-02	Prozesse und Sitzungen anzeigen und beenden	Should	Should	M	F-2: KILL QUERY und pg_cancel_backend belegt	Test: beendete Sitzung ist weg
W-03	Variablen und Status lesen	Could	Could	S	—	Durchlauf

Nr.	Anforderung	CR	Plan	Grösse	Feasibility / Rückfall	Abnahme (Kurzform, vollständig im CR)
W-04	Datenbank-Benutzer, Rollen und Rechte geführt verwalten	Should	Should	L	—	Tests je Treiber (ohne SQLite)
W-05	Wartung	Could	Could	S	—	Durchlauf

TB 10 — Weitere Objekte, Analyse, Sicherung → v1.1 (Stufe 3; 7 Anforderungen, 18 Punkte, Must 0)

Nr.	Anforderung	CR	Plan	Grösse	Feasibility / Rückfall	Abnahme (Kurzform, vollständig im CR)
D-08	Suchen und Ersetzen	Could	Could	M	—	Test je Treiber
S-06	Views, Routinen, Trigger, Sequenzen, Typen und Enums	Should	Should	XL	—	Test je Objektart und Treiber
S-07	Beziehungsdiagramm aus den Fremdschlüsseln, als SVG exportierbar	Should	Should	L	—	Durchlauf mit Basalto-Datenbank
Q-05	Ergebnisse bearbeiten, wenn die Abfrage eine Tabelle mit Primärschlüssel liest	Could	Could	M	—	Test
Q-06	EXPLAIN als Baum mit Hinweisen	Should	Should	M	—	Durchlauf je Treiber
E-03	Sicherung und Wiederherstellung über die Werkzeuge des Servers	Should	Should	L	—	Test überspringt sichtbar, wenn das Werkzeug fehlt
E-04	Export als Excel	Could	Could	S	—	Test

TB 11 — Komfort: Schemavergleich, Laravel, Herd, SSH (Stufe 4; 4 Anforderungen, 10 Punkte, Must 0)

Nr.	Anforderung	CR	Plan	Grösse	Feasibility / Rückfall	Abnahme (Kurzform, vollständig im CR)
V-07	Herd-Dienste und .env der Herd-Sites als Vorschläge erkennen	Could	Could	M	—	Durchlauf auf Hoangs Mac
V-08	SSH-Tunnel	Could	Could	L	—	Test gegen einen lokalen SSH-Dienst
S-08	Schemavergleich zweier Datenbanken mit Änderungsskript	Could	Could	L	—	Test mit zwei Ständen
S-09	Laravel-Migration aus einer bestehenden Tabelle erzeugen	Could	Could	M	—	Test: Migration erzeugt dieselbe Tabelle auf einem anderen Treiber

TB 12 — Team und KI → v1.2 (Stufe 4; 2 Anforderungen, 4 Punkte, Must 0)

Nr.	Anforderung	CR	Plan	Grösse	Feasibility / Rückfall	Abnahme (Kurzform, vollständig im CR)
A-08	Aktive Sitzungen sehen und beenden	Could	Could	M	—	Test: beendete Sitzung ist abgemeldet

Nr.	Anforderung	CR	Plan	Grösse	Feasibility / Rückfall	Abnahme (Kurzform, vollständig im CR)
Q-08	SQL-Hilfe mit KI — nur eingeschaltet, nur mit eigenem Schlüssel, nur das Schema wird gesendet	Could	Could	M	—	Test mit Attrappe: keine Datenzeilen im Auftrag

3. Regeln

- **Must im Plan** trägt den Meilenstein der Stufe (Alpha, v1.0, v1.1, v1.2); **Should** ↓ ist im CR Must, hat aber einen Rückfallweg und darf bei Zeitnot in die nächste Stufe rutschen — der CR bleibt unverändert, die Verschiebung steht im Timebox-Protokoll.
- Grösse ist eine relative Schätzung; die Personentage je Timebox ergeben sich aus dem Budget der Stufe anteilig nach Punkten — der Timebox-Plan übernimmt sie.
- Das Skript prüft, dass jede Anforderung genau eine Timebox in ihrer Stufe hat; neue Anforderungen im CR brechen den Lauf, bis sie zugeordnet sind.
- Änderungen an Zuordnung oder Grösse nur im Skript; danach `python3 dokumentationen/werkzeuge/prl-bauen.py` und das PDF neu bauen.

Anhang F — Timebox-Plan — phpdbadmin

Stand: 28.10.2026 · v1.0 (Foundation). Zuordnung der Anforderungen und Punkte aus der PRL (erzeugt), Personentage anteilig aus dem Budget je Stufe (obere Schätzung 53 PT). Wireframe-Bereiche siehe mockups.

phpdbadmin — Timebox-Plan

Dreizehn Timeboxes in fünf Stufen; Personentage aus der PRL (obere Schätzung 53 PT). Reihenfolge, keine Termine (neben Basalto, R-03)



Must-Anteil im Plan (PRL, nach Punkten)

Stufe 0: 90 % (Fundament ohne Rückfallweg, bewusst) · Stufe 1: 57 % · Stufe 2: 44 % · bis v1.0 zusammen: 58 % · Stufen 3 und 4: 0 %

Neun Must-Anforderungen mit Rückfallweg laufen im Plan als «Should ↓» (z. B. Befehlspalette → Navigator, Import → SQL-Datei im Editor)

1. Die dreizehn Timeboxes

TB	Inhalt	Anforderungen (Must im Plan fett)	Wireframes	PT	Braucht
00	**Fundament:** Starter-Kit bereinigen (Passwort, Dashboard, Profil weg), Code und Passkey, App-Datenbank, Verbindungen anlegen und testen, erlaubte Hosts, Dialekt-Schicht, Tests auf vier Datenbanken	**A-01–A-05**, A-06 ↓, A-09, **V-01–V-03, V-05**, **B-01–B-03, B-06, B-08**	1–4, 13	~8,0	Foundation ✓
01	Katalog und Navigator, Strukturansicht, Umgebung und Schreibschutz, Kontowarnung	**O-01, O-04, S-01, V-04, N-06**, V-06, V-10, O-06 ↓, N-04 ↓	3, 5, 8	~4,0	TB 00
02	Datenraster und Filter, Fremdschlüssel, Zählen, Reiter, Zustände	**D-01, D-02, N-01**, D-03, D-06, O-03, O-05, O-09 ↓	5, 7	~3,6	TB 01
03	SQL-Editor, Grenzen, Abbrechen, Verlauf, SQLite-Arbeitsprozess, Befehlspalette → **Alpha**	**Q-01–Q-03**, Q-04, Q-09, O-02 ↓	10, 12	~3,4	TB 01
04	Bearbeiten mit Vorschau, Rückgängig, Bestätigung gefährlicher Anweisungen	**D-04, N-03, Q-07**, D-09, E-05, V-11, D-05 ↓, D-07 ↓	5, 6, 7	~4,7	TB 02, TB 03
05	Struktur ändern: anlegen, ändern, je Treiber anwenden	**S-02, S-03**, S-05, S-04 ↓	9	~2,7	TB 01, TB 04
06	Export und Import im Hintergrund, Statistik	**E-01**, E-06, B-07, E-02 ↓	11	~2,0	TB 02
07	Konten, Rechte je Verbindung, Teilen, Protokoll mit Maskierung	**B-04**, A-07, V-09	14	~1,8	TB 00, TB 04
08	Sicherheitsprüfung, Barrierefreiheit, Mobil, Hilfe, Installation → **v1.0**	**N-02**, O-07, O-08, N-05, B-05	alle	~2,7	TB 04–07
09	Server-Verwaltung: Übersicht, Prozesse, Variablen, Benutzer und Rechte, Wartung	W-01–W-05	— (in TB 09 entworfen)	~3,3	v1.0
10	Views, Routinen, Trigger, Beziehungsdiagramm, EXPLAIN, Ergebnisse bearbeiten, Suche, Sicherung, Excel → **v1.1**	S-06, S-07, Q-05, Q-06, D-08, E-03, E-04	—	~6,7	v1.0
11	Schemavergleich, Laravel-Migration, Herd-Erkennung, SSH-Tunnel	S-08, S-09, V-07, V-08	—	~7,1	v1.1
12	Sitzungen verwalten, SQL-Hilfe mit KI → **v1.2**	A-08, Q-08	—	~2,9	v1.1

↓ = im CR Must, im Plan Should mit Rückfallweg (PRL § 2).

2. TB 00 in Scheiben

> **Abgeschlossen am 28.10.2026** — Protokoll. Ergänzt auf Hoangs Wunsch um die Ersteinrichtung (Link aus `phpdbadmin:install`) und das Passkey-Onboarding nach der ersten Anmeldung.

#	Scheibe	Inhalt	Anforderungen
1	Starter-Kit bereinigen	Passwort-Anmeldung, Registrierung, Passwort-Reset und -Bestätigung, Zwei-Faktor, E-Mail-Bestätigung, Dashboard, Profil, «Konto löschen» entfernt; Fortify nur für Passkeys ohne eigene Routen (kein Passwort-Endpunkt); Spalten und Tabelle für Passwörter weg; Startseite = Verbindungsübersicht (leer); Kontomenü mit Sicherheit und Darstellung; Tests des Starter-Kits durch eigene ersetzt; `APP_URL=https://phpdbadmin.test`, `APP_LOCALE=de`	A-01, A-04, A-05, B-01
2	Anmeldung	Code per Mail (HMAC, 10 Minuten, einmalig, benannte Limiter, gleiche Antwort für Unbekannte), Passkey anmelden und verwalten, `phpdbadmin:install` für das erste Konto, Einladung mit Ablauf, Anmeldungen im Protokoll; Software-Authenticator für Tests nach dem Muster von Basalto	A-02, A-03, A-06, A-09
3	Verbindungen	Migrationen nach dem Datenmodell, Verbindung anlegen und bearbeiten für vier Treiber, «Verbindung testen» mit Klartext je Treiber, Zugangsdaten verschlüsselt und nie im Browser, erlaubte Hosts und SQLite-Ordner (`realpath`, einmal auflösen)	V-01, V-02, V-03, V-05, B-06
4	Dialekt-Schicht	Vertrag und vier Umsetzungen (Katalog-Grundlagen, Zeitlimits, Schreibschutz-Wächter, Fehlerzuordnung, Kontorechte); Proben aus `tests/Feature/Feasibility` zu Vertragstests umgebaut; `composer.json` auf PHP `^8.5` (ADR-011)	B-02
5	Qualität und Abnahme	lokales Skript für die vier Treiber, Workflow sparsam (ein Auftrag je Push, Matrix wöchentlich), Probe-Seiten übernehmen oder entfernen, Durchlauf im Browser gegen die Wireframe-Bereiche 1–4 und 13 bei 1440 und 375 px, Protokoll	B-03, B-08

3. Meilensteine und Abnahme

Meilenstein	nach	Abnahme
Alpha	TB 03	Hoang nutzt phpdbadmin eine Woche zum Nachsehen statt phpMyAdmin/TablePlus; Vertragstests auf vier Treibern grün; Navigator mit 1'000 Tabellen und Raster mit 1 Mio. Zeilen im Browser flüssig
v1.0	TB 08	Kernabläufe (Bearbeiten, Struktur, Export, Import) auf vier Treibern getestet, inklusive Elterntabelle mit Kindern; White-Box-Sicherheitsprüfung nach OWASP ohne hohen Befund; axe ohne Befund in den Kernbildschirmen; Installation in einem leeren Ordner unter 15 Minuten
v1.1	TB 10	Server-Verwaltung auf drei Server-Treibern getestet; Sicherung und Wiederherstellung mit den Server-Werkzeugen
v1.2	TB 12	Schemavergleich und Migration gegenseitig zwischen zwei Treibern; KI-Hilfe nur mit Schema (Test mit Attrappe)

4. Reihenfolge und Abhängigkeiten

> **Umsetzung ab 28.10.2026 abends** (Hoangs Auftrag «mache alle TB durch»): Timeboxes ohne gegenseitige Abhängigkeit laufen parallel in eigenen Arbeitskopien mit eigener Testdatenbank (`PHPDBADMIN_TEST_DATABASE`) — Server-Verwaltung (TB 09), Export und Import (TB 06), Konten, Rechte und Protokoll (TB 07, mit V-06) — und werden nach Prüfung zusammengeführt; der Hauptstrang folgt der Reihenfolge TB 01 → 02 → 03 → 04 → 05 → 08. Stand je Timebox im README der Phase 04.

TB 00 trägt alles. TB 02 (Raster) und TB 03 (Editor) hängen beide nur an TB 01 und können in beliebiger Reihenfolge folgen; Bearbeiten (TB 04) braucht beide, weil die Vorschau im Editor geöffnet werden kann. Rechte und Protokoll (TB 07) kommen nach dem Bearbeiten, weil erst dann schreibende Aktionen

protokolliert werden. Die Sicherheitsprüfung (TB 08) steht vor v1.0, nicht danach. phpdbadmin läuft neben Basalto (R-03): eine Timebox wird angefangen und fertig gemacht, nicht unterbrochen.

5. Was aus der Feasibility übernommen wird

Aus	Wird
`tests/Feature/Feasibility/ProbeTestCase`	Basis der Vertragstests (Verbindungen zu den Herd-Diensten, ausdrückliches Überspringen)
`CatalogProbeTest`	Vertragstests des Katalogs (TB 00/01)
`DdlPreviewProbeTest`	Abnahmetests von S-02, S-03, N-03 (TB 05)
`ConnectionProbeTest`	Vertragstests für Zeitlimit, Abbrechen, Fehlerzuordnung (TB 00/03)
`SecurityProbeTest`	Abnahmetests von V-04, V-05, V-10, B-04 (TB 00/01/07)
`LargeTableProbeTest`	Leistungsprobe N-01, E-01 (bleibt hinter `PROBE_GROSS=1`)
`routes/probe.php`, `pages/probe/*`	Raster und Editor als Ausgangspunkt für TB 02/03, danach entfernt

6. Protokoll je Timebox (Vorlage)

```
# TB nn — Titel
## Ziel (ein Satz)
## Umfang: Anforderungen aus der PRL (Must / Should ↓), Wireframe-Bereiche
## Abnahmekriterien (aus CR-001 § 4, konkret je Anforderung)
## Abweichungen von den Wireframes (benannt, Wireframes nachgezogen)
## Tests (neue und geänderte; je Treiber; zweites Konto für Rechte)
## Commits
## Gelernt (Lektionen in den Vault, Projektspezifisches ins Memory)
```